Content Atelier AI Processing Architecture

End to end design for ChatGPT Plus scheduled tasks custom MCP and skills

14 October 2026 | Architecture version 1.0 | Proposed pilot and production path

Content Atelier should remain the workspace where Whitney, Giselle, and the WIA team review opportunities, steer revisions, approve exact versions, and prepare production. A connected ChatGPT Plus account can be evaluated as a background executor through scheduled tasks, a custom MCP plugin, and editorial skills. The Content Atelier backend should own sources, eligibility, permissions, validation, durable memory, and every workflow transition.

The 14 October test establishes that this particular Plus account and plugin configuration completed one scheduled MCP write without interactive approval. It does not establish that the complete content pipeline, recurring reliability, project memory, or production permissions work unattended. The recommended next step is a bounded source to angle to revision pilot, followed by an accepted brief and only then a real pack workflow.

PostgreSQL remains the authoritative memory and workflow store. Accumulated ChatGPT context can help recover Whitney's preferences and calibrate voice, but required instructions must be retrieved explicitly for each job. A versioned preference.md should be a reviewed projection of backend records. Project-only memory is valuable for contained editorial conversations; the documented restriction on Work in such projects prevents assuming it can host this executor today.

This document defines the proposed operating model, detailed flows, application contracts, controls, migration approach, and acceptance criteria. Engineering can use it to implement a pilot; Whitney can use it to assess editorial ownership and review effort. Proposed defaults require configuration and verification before activation.

Document navigation

The opening sections establish evidence and decisions. Sections 4 to 7 explain system boundaries and memory. Sections 8 to 14 describe the complete editorial journey. Sections 15 to 19 define events, jobs, tools, and validation. Sections 20 to 24 cover security, scheduling, operations, migration, and rollout. Appendices supply examples, adversarial cases, and sources.

Section	Subject
1 to 3	Evidence baseline and architectural decisions
4 to 7	Components context personalization and persistent preferences
8 to 14	Intake discovery gates revisions briefs packs and actual use
15 to 19	Exact event job MCP and validation contracts
20 to 24	Security scheduling recovery implementation and pilot roadmap
Appendices A to D	Worked example contract catalogue acceptance tests and sources

1 Evidence baseline and interpretation

Evidence labels

Use four labels when assessing the architecture. OBSERVED means a dated experiment recorded actual behavior. REPORTED IMPLEMENTATION means the agenda or implementation record describes existing code or previous verification. DOCUMENTED means a current primary OpenAI page describes a product capability. PROPOSED means this document specifies behavior still to build or verify. An OPEN item requires an operating decision or account-specific test.

These labels avoid conflating a successful tool call, implemented validation, and editorial acceptance. None implies the others. Source identifiers in square brackets resolve to Appendix D.

What the MCP experiment established

The experiment used a standalone Python MCP server, SQLite receipts, and a temporary Cloudflare Quick Tunnel. It did not access or modify Content Atelier. Local and remote official MCP clients discovered tools and verified writes. The manual ChatGPT-originated receipt manual-001 was independently read from the backend at 06:09:57.130Z, or 11:39:57.130 Asia/Kolkata. [S2]

The initial scheduling request created no task. ChatGPT reportedly required a successful read-only plugin access check, while the initial server exposed only a write tool. Adding check_connection, refreshing the plugin, and retrying produced a task for 11:53:03 Asia/Kolkata. This preflight explanation is account-specific evidence; it is not a universal documented rule. [S2]

The server recorded scheduled-001 at 06:24:01.690Z, or 11:54:01.690 Asia/Kolkata, 58.690 seconds after its scheduled time. The user had closed the ChatGPT tab. Independent receipt inspection preceded the supplied completion screenshot, which showed no interactive approval and duplicate false. Full browser-process exit was not tested. [S2]

Test area	Observed result	Architectural implication
Local and remote MCP	Tool discovery and execution passed	The transport can bridge a hosted ChatGPT task and a backend
Manual and scheduled writes	One real receipt for each ChatGPT path	A bounded unattended pilot is justified on this account
Persistence	Receipt survived server restart	Persistence was tested in SQLite only
Duplicate and conflicting IDs	Identical retry reused timestamp; conflict failed	Port the invariant to PostgreSQL with unique keys and canonical hashes
Eight concurrent identical calls	One receipt and eight write audit events	Distinguish one business result from repeated delivery attempts
Input boundaries	Invalid IDs and messages rejected; 2000 characters accepted	Validate schemas and semantic boundaries server-side
HTTP protections	Malformed JSON 400; over 64 KiB 413; Host 421; Origin 403	Preserve defensive ingress checks, then add production authentication

Test area	Observed result	Architectural implication
Transport compatibility	JSON POST; GET mcp 405; stateless DELETE checks; legacy version passed	Negotiate supported protocol; do not copy a legacy-only transport into an events implementation
Infrastructure lifecycle	Backend and temporary tunnel stopped afterward	The former hostname is inactive and is not a deployment endpoint

One late successful run cannot estimate an on-time percentile, throughput, retry behavior, multi-step reasoning quality, or long-term permission stability. Test results also do not demonstrate OAuth, tenant isolation, real transcripts, skills activation, or a recurring weekly cycle. Treat the standalone test as connectivity and write evidence.

What the project agenda reports

The 14 October agenda describes a React/TypeScript frontend with mock and live modes and a Python/FastAPI backend using PostgreSQL. Its SQL workflow already implements source admission, moment and angle generation, historical comparison, researched briefs, and approval controls through Gate 2. It reports partial or failed extractions and two generated brief pilots that remain blocked at Gate 2. [S1, S4]

The baseline uses thirteen tables and two persistent vector columns for transcript chunks and angles. Existing jobs have locks, leases, heartbeats, events, and explicit retries. Source-byte archives, exact requests, raw responses, caches, costs, and dossiers are linked file artifacts. Redis/RQ is not the executor of this SQL path. [S1, S3]

The Weekly Studio demonstrates broader progress using saved content and browser-local state. Choosing live data mode exposes real angle and brief review surfaces; it does not make every mock screen durable. Automated SQL-backed packs, production delivery, and confirmed external publishing remain future work. The latest fuller handbook profile is separately versioned and has not replaced production defaults. [S1]

Current product documentation and unresolved boundaries

DOCUMENTED: Plugins can combine editorial skills and MCP tools and return structured results without a custom ChatGPT interface. Skills provide instructions; MCP supplies live information and controlled actions. Skill selection is a model behavior, so this design also makes required workflow and contract versions explicit in job context. [S5, S6]

DOCUMENTED: The Help Center describes eligible paid recurring tasks up to hourly and pauses when required action approval is pending. It also says project-created tasks cannot access uploaded or stored project files. [S7]

DOCUMENTED: ChatGPT Learn distinguishes standalone tasks from tasks returning to an existing chat. It describes skills and plugins in scheduled Work and minute-based follow-up loops in chats. This broader cross-surface guidance differs from the Help Center's cadence and project-file guidance. The pilot therefore uses the tested cloud Work path, hourly polling, and MCP retrieval; account-specific tests must settle any shorter cadence or file access. [S8]

DOCUMENTED: Project-only memory excludes outside conversations and saved personal memories, and Work is unavailable in the project. A default-memory Plus project may reference context beyond its project. A project name therefore does not provide memory isolation. [S9]

DOCUMENTED: The newer MCP Events guide describes custom webhook subscriptions in Work, requiring protocol version 2026-07-28, persistent subscriptions, and verified callbacks. This means the

earlier architecture's blanket assumption that custom backend events cannot initiate work should be revised. Custom event execution on this Plus account remains untested. Preserve polling as the pilot baseline. [S11]

2 Product purpose and editorial requirements

Content Atelier turns the useful thinking in WIA's conversations into distinct opportunities, researched briefs, and approved production material. Its audience includes venue owners, photographers, planners, videographers, and other wedding professionals. The source participant and intended content audience can differ; broaden relevance only while retaining the original insight and support. [S1]

The editorial question is what a conversation reveals that helps a wedding business owner understand WIA's perspective, value its expertise, or see how its work could help them. A useful direction can show collaboration, a diagnostic question, positioning judgment, or a process without a dramatic measured result. Accurate summarization alone is insufficient.

Editorial requirement	Operational expression
ATTRACT	Reach suitable new audiences with supported perspectives and useful interest
WARM	Build trust in WIA's reasoning processes collaboration and expertise
CONVERT	Explain offer fit and value using properly scoped service facts or proof
Founder voice	Calm confident warm curious specific; respectful of experienced owners
Offer context	Verify current ELEVATE Ads Accelerators and LeadLock scope before making promises
Attribution	Whitney's first person requires her actual perspective or experience; team work can use we
Distinction	A different hook title client name or pillar does not make a new central point
Evidence	Preserve proposals uncertainty qualification units attribution and timeframes
Output mix	No forced equal pillar split or angle quota; pack capacity is a later decision

Finished material may follow Hook, Problems/Desires/Symptoms, Main Point, Transformation, and CTA when suitable. Transformation can be a change in understanding. Captions should add useful context while staying aligned. The agenda's proposed pack defaults are one email, a seven to ten slide carousel, one to three reels, and long and short captions. Confirm actual assignments and capacity before generating assets. [S1]

Success means Whitney approves worthwhile directions with little strategic rewriting and Giselle receives usable production guidance. Measure opportunities missed, unsupported claims, attribution errors, repeated arguments, time spent reviewing, and production rewriting. The earlier five to ten minute weekly review aspiration is a target to evaluate, not an achieved service level. Operational correctness and editorial usefulness need separate scorecards.

3 Architecture decisions and scope

Preserve the selected architecture

The user-selected architecture assigns all interactions, editing, comparisons, approvals, and status displays to Content Atelier. The backend holds source storage, jobs, approvals, validation, versioning, memory, and state. MCP provides an authorized HTTPS interface. ChatGPT plus skills handles understanding, angle discovery, research, generation, and revisions. Schedulers periodically check eligible backend work. [S0]

The selected governing principle is preserved: The backend decides WHAT needs to happen. ChatGPT decides HOW to generate the requested content. The frontend lets users decide whether the result is acceptable.

Decision	Proposed choice	Why it matters
Product interface	All editorial work in Content Atelier	Review and state remain available without opening ChatGPT
Execution	Cloud ChatGPT Work account with Plus pilot	Builds on observed scheduled MCP execution
Truth and memory	PostgreSQL records plus immutable linked artifacts	State and provenance survive unrelated chats and executor changes
Job ownership	Backend eligibility and atomic reservation	Concurrent schedulers cannot choose unauthorized next steps
Approvals	Human decisions on exact hashes and saved records	New content cannot inherit old approval silently
Skills	Versioned procedures with live context retrieval	Guidance remains maintainable without embedding dynamic client memory in packages
Cadence	Weekly discovery and hourly editorial worker	Matches selected design and conservative documented paid cadence
Delivery	Draft storage and human handoff	Editorial approval does not become an external publication action
Faster execution	Separate optional MCP Events pilot	New documented possibility requires its own evidence

Users should not need ChatGPT for normal editorial work. An operator may still need it to connect accounts, approve app access, create or repair schedules, update a retired model, or clear a host-side pause. ChatGPT may retain internal chats, task history, and notifications. The product can hide these from the editorial journey but cannot promise that they do not exist.

The document is an implementation specification, not an instruction to activate schedules, alter permissions, run paid calls, connect production sources, or approve content. Keep current external-call switches closed until the bounded pilot is authorized through the project's operating controls.

4 System components and trust boundaries

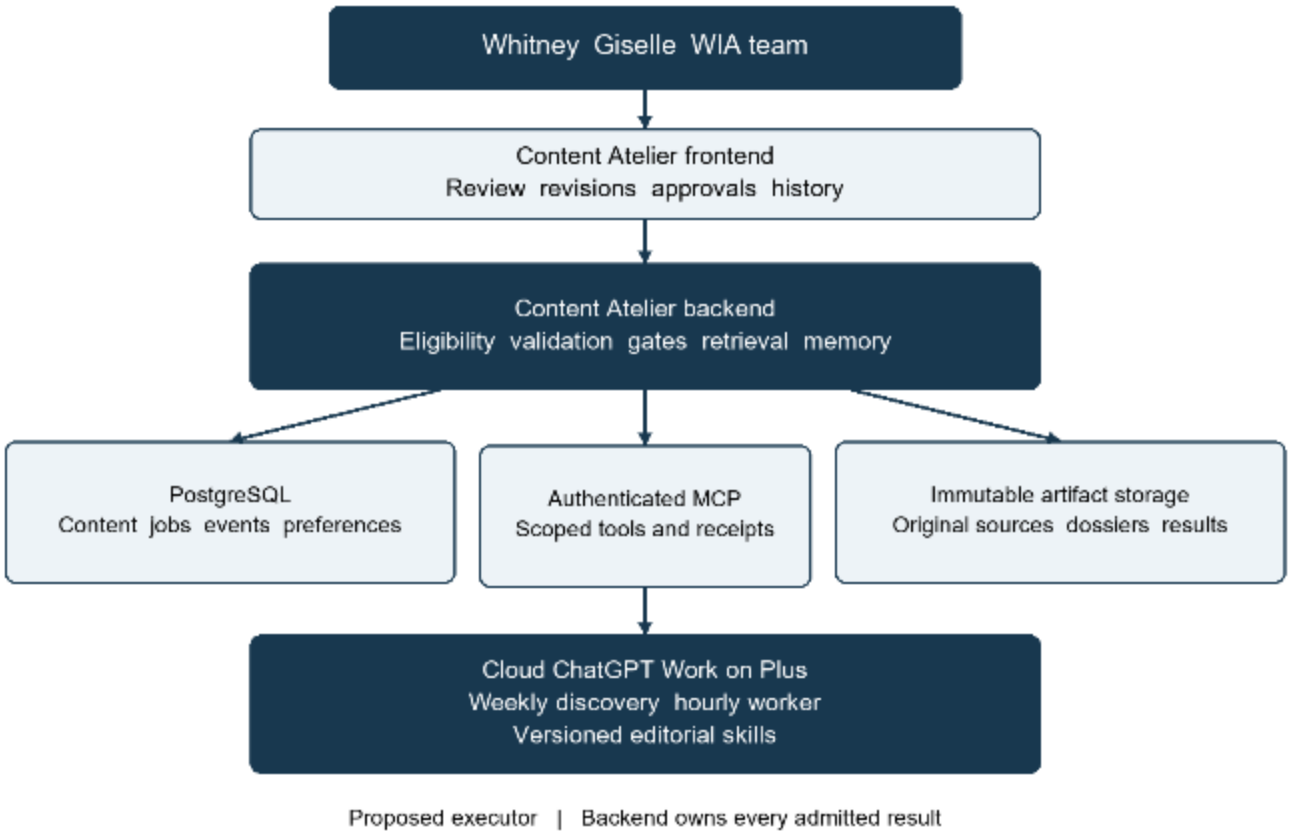


Figure 1 Proposed system architecture

Content Atelier services

The browser uses the protected frontend server and its existing sign-in integration. Backend endpoints validate the authenticated actor's capabilities and tenant. The frontend shows committed SQL records, never parses a ChatGPT conversation as the data channel. Server-sent events or application polling can refresh status after database commit.

The FastAPI backend contains domain logic, validators, intake, retrieval, and workflow eligibility. The MCP adapter should call these services rather than duplicating rules or directly permitting arbitrary SQL. Existing local workers continue deterministic normalization, validation, embedding where approved, comparison, export, and recovery. ChatGPT is a proposed executor for selected generative stages.

A production MCP endpoint needs a stable HTTPS hostname, authenticated access, bounded requests, health monitoring, and a deployed backend reachable while operator devices are offline. Object or artifact storage preserves original bytes, canonical text, dossiers, request/result envelopes, and exports. PostgreSQL records hashes and resolvable references; object writes must be durable before referenced business rows commit.

Executor ownership

Mark every generative intent with executor_kind, such as existing_provider or chatgpt_plus. Only the configured executor can claim it. The current provider worker must exclude ChatGPT-assigned jobs; the MCP worker must exclude existing-provider jobs. Both may use the same repository transition

methods. This avoids double generation and two sources of result identity.

ChatGPT executes one bounded intent using tools and the named skill. It receives no database credential, provider key, arbitrary shell access, or approval capability. It can submit candidates and issue reports. The backend decides whether to admit them and whether an eligible next job exists.

Boundary enforcement

Treat source transcripts, web pages, past chats, feedback bodies, and generated prose as untrusted content. They may contain instructions unrelated to the job. Put workflow authority in authenticated server policy and reviewed skill instructions, and keep source material clearly marked as data. The model should ignore embedded instructions; server constraints must still prevent unauthorized reads, approvals, exports, and writes if it fails to do so.

The WIA tenant in PostgreSQL, the connected MCP principal, and a ChatGPT project are different boundaries. Backend tenant isolation does not prevent a personalized shared executor from recalling another client's material. Start with one client and one narrowly scoped executor identity. Assess separate execution accounts or supported organizational identities before adding clients; do not share credentials to create a service account.

5 PostgreSQL and immutable business memory

Reuse the thirteen table baseline

The schema baseline has weeks, transcripts, transcript_chunks, key_moments, content_angles, angle_moments, content_briefs, content_packs, content_assets, calendar_entries, pipeline_runs, pipeline_jobs, and pipeline_events. Preserve these relationships and their hash-bound approval semantics. A table existing for packs or calendar does not establish its workflow. [S3]

Store	Authority and contents
transcripts and transcript_chunks	Immutable original text passage map metadata and searchable chunks
key_moments and angle_moments	Validated propositions and exact source roles linked to angles
content_angles	Saved central points audiences pillars comparison outputs CTA and Gate 1 decision
content_briefs	Exact consumed angle hash dossier references claims questions and Gate 2 decision
content_packs and content_assets	Ordered planned slots exact governed copy checks and Gate 3 decision
calendar_entries	Exact asset version intended schedule and actual publication confirmation
pipeline_runs and pipeline_jobs	Frozen inputs attempts ownership eligibility progress and results
pipeline_events	Compact meaningful milestones feedback decisions and failures
Linked artifacts	Source bytes raw requests results dossiers caches exports and execution receipts

Source evidence is distinct from historical content. Vector retrieval locates relevant candidates; exact original passages establish what was said. Angle embeddings compare argument similarity; they cannot support a factual claim about a client. Preserve the two vector(1536) contracts and configured model compatibility. Query vectors can remain transient artifacts. A ChatGPT executor does not automatically supply replacement embeddings.

Once approved or consumed by downstream work, preserve the existing row and create a same-table replacement linked by supersedes_id. Use revision as an external logical version in the adapter; map it to an actual row ID and content hash. Do not replace this compact schema with a new revision registry. Preserve first_generated_at across the same logical angle family; a replay does not make an old argument new.

Argument edits invalidate matching, relevant scores and visibility until reassessment. Clear embeddings only when their canonical semantic input changes. CTA and evidence changes may invalidate approval even when the embedding input is unchanged. Hash every approval-relevant field and referenced assignment, not only visible copy.

Proposed memory extension

Add a reviewed editorial_preferences store only through a separate migration. Proposed records contain tenant, logical rule identity, immutable version, status, structured scope, wording, examples, provenance, confidence, validity window, approver, supersession, and hash. Store exact feedback in append-only pipeline_events with a dedicated event type or a purpose-specific store if access/query requirements justify it. The schema decision must be explicit; do not imply these records already exist.

Durable binary artifacts need a database manifest or strongly governed references, backup, encryption, and tenant access checks. Write an artifact to a content-addressed location, verify its digest, then commit its reference. Clean orphaned artifacts only after a retention grace period. Hashes detect change but do not replace authorization or backup.

Memory retrieval

Each job builds a frozen packet with required policy, current verified facts, source evidence, previous revisions, exact relevant feedback, approved exemplars, prior arguments, and actual use. Store retrieval query/configuration hashes and inclusion/omission reasons. A missing history index means unknown comparison coverage, not no repetition.

Retrieve by purpose, tenant, audience, offer, format, campaign, recency, and semantic relevance. Mandatory current policy and original grounding remain present even if a relevance ranking would omit them. Truncate optional history transparently with omission counts and an explicit gap. If mandatory context cannot fit, block the request or use a separately approved processing policy.

6 Accumulated ChatGPT context and personalization

What can be available

Whitney's conversations in the same executing account may influence responses when memory and history reference are enabled and relevant. Current-chat context may persist in a task returning to that chat. Other-account history, another member's private context, and the complete history of every prior task do not become available merely because a plugin is installed. There is no established complete memory export through Content Atelier's MCP server. [S8, S10]

Context layer	Possible use	Requirement for this design
Current chat history	Ongoing editorial reasoning and earlier examples	Pin required information in job context rather than relying on recall
Saved memory and past chats	Persona preferences and useful continuity when available	Treat as advisory; detect conflicts with current authenticated policy
Project instructions	Editorial defaults for chats in that project	Do not make them the only source of scheduled instructions
Project-only memory	Contains conversations within its project	Use for editorial curation while Work restriction remains
Skill package	Repeatable steps and static examples	Version the procedure and fetch dynamic preference.md
Backend context	Verified client facts feedback evidence and content history	Mandatory authority for every executable intent

DOCUMENTED: Memory does not retain every conversation detail and selects relevant available information. Settings and source visibility vary; a memory summary is not a complete provenance record. Turning memory off does not delete past chats. [S10]

A scheduled run can use information from preceding runs when that product path retains it, but this cannot replace explicit persistence. A successful revision learned in ChatGPT must become a backend feedback and decision record before future jobs depend on it. The model should not claim that an unsaved preference is authoritative.

Respect the preference for a contained project

A dedicated WIA project with project-only memory is appropriate for human editorial calibration and knowledge curation. Move or copy only deliberately selected client conversations, review their meaning, and turn stable conclusions into backend records. Ordinary saved memories outside that project are excluded by the documented project-only rules. [S9]

The executor cannot currently be assumed to run as Work inside that project. The scheduled-task documentation also excludes files stored in the project from project-created tasks. Thus project containment, Work availability, and scheduled retrieval are separate tests. Do not assume uploading preference.md into a project feeds future unattended execution. [S7, S9]

Use a contained project as the editorial workshop, and a compatible cloud Work chat as the execution path with explicit MCP context. If future account tests demonstrate supported project-only Work and scheduled tool access, reconsider placement; keep backend authority and pinned versions unchanged. A default-memory project is an option for compatible Work, but it is not project-only isolation on Plus.

Recover accumulated knowledge without losing it

Inventory selected conversations, approved scripts, founder notes, current offer records, and existing guides. Ask for a reviewed synthesis of persona, audience, wording preferences, examples, prohibitions, and unresolved conflicts. Preserve source chat/message references when available and mark recalled-only items as unverified candidates.

Whitney approves the proposed reusable rules through Content Atelier. Import only relevant business context, and resolve stale examples against her latest explicit instructions. Test the extracted guide on

previously unseen recordings. This captures valuable accumulated learning without treating a model's recollection as a complete factual archive.

Conflict policy

Apply separate hierarchies for instruction and evidence. For editorial direction, the latest authorized scoped Whitney instruction wins over older preferences; current verified strategy/offer records follow; then active approved guidance, historical approved examples, and model suggestions. Exact feedback controls the targeted revision within its scope. Source evidence remains authoritative about that exchange. A stylistic instruction cannot turn a proposal into an implemented outcome.

Enforce security and source-use restrictions before editorial direction. A reviewer can request different framing but cannot make an unverified booking figure true. Show conflicts and missing support in Content Atelier. An optional recalled detail may be proposed for review; it cannot change audience, offer scope, ownership, or approved facts silently.

7 Versioned preference md and the learning loop

A reviewed projection

Use the exact preference.md filename as the model-facing projection of active editorial rules. PostgreSQL stores structured rule versions and their provenance. The backend renders immutable snapshots, assigns preference_version and SHA-256 digest, and serves them through get_job_context or get_preference_snapshot. A skill can contain an example or fallback guide, but it must instruct the executor to retrieve the job's exact live snapshot.

An imported MCP skill can be a static package snapshot; updating its source file does not necessarily refresh a deployed plugin. Keep changing client preferences in backend reads, and roll out procedure changes as explicit plugin/skill releases. [S13]

Lifecycle and promotion

Feedback is stored verbatim with entity, base row/hash, actor, scope, time, resolution, and reuse eligibility. A generation skill can propose a candidate rule with supporting decisions and counterexamples. Approval of an angle indicates direction acceptance; it does not prove every word or inference is a universal preference.

Use CANDIDATE, ACTIVE, SUPERSEDED, REVOKED, and EXPIRED statuses. Only an authorized editorial owner can activate or revoke a reusable rule. Proposed default: a repeated pattern from three independently reviewed examples can enter the promotion queue; a single explicit instruction can also enter with its original scope. Neither path auto-activates. Confidence ranks review candidates rather than replacing human authority.

Separate stable voice preferences, strategic selection principles, format rules, temporary campaigns, current offer facts, and case-specific restrictions. Scope can include tenant, audience, pillar, offer, format, campaign, and action. Avoid treating a correction about one generic tip as a ban on technical education. Expire campaign rules and facts requiring verification on their review date.

Reusable-rule review is memory governance rather than a fourth content gate. Batch candidate reviews periodically and measure the added review effort. Explicit structured preference instructions can be approved in the same authenticated editorial action when the UI clearly identifies their reusable scope.

Rule field	Exact meaning
rule_id and version	Logical identity and immutable positive integer version
status	Lifecycle state; only ACTIVE unexpired versions enter normal packets
scope	Structured audience pillar offer format campaign and action filters
wording and rationale	The approved rule and reason for applying it
examples and counterexamples	Reviewed pointers explaining the intended boundary
provenance	Exact feedback event and reviewed entity/hash references
approved_by and approved_at	Trusted editorial identity and server time
valid_from and valid_until	Applicable window; null end means no fixed expiry
supersedes and content_hash	Previous version and canonical digest

Pinning and rollback

Every job pins a preference snapshot alongside strategy, brand, offer, skill, and validator versions. Do not hot-swap preferences mid-generation. A new preference generally affects future jobs; a material correction can explicitly cancel and supersede in-flight work. Safety or clearance revocation takes immediate effect and causes commit-time revalidation.

Render snapshots deterministically from the active structured rules. Publish the new active pointer and projection atomically after approval. If publication fails, continue serving the last verified active version and flag the discrepancy. Roll back by selecting a reviewed earlier snapshot as a new activation event, without deleting later versions or rewriting past job provenance.

Example preference snapshot

This is an illustrative proposed schema and rule, not a newly approved Whitney preference.

```
---
tenant: wia
preference_version: 12
status: ACTIVE
effective_at: 2026-10-14T07:00:00Z
strategy_version: wia-strategy-2026-10
render_contract: preference-md-v1
---

Rule PREF-004 version 3
Scope: WARM process stories for wedding business owners
Guidance: Show the questions and reasoning WIA uses with a client.
Boundary: Technical education remains suitable when it serves a clear point.
Provenance: reviewed feedback and approved angle references
Example: Explain how the team helps a client assess imagery choices.
Avoid: Claiming imagery increased bookings without verified evidence.
```

Rule IDs aid review; return the digest outside the hashed file.

8 Source intake and weekly preparation

Stage 0 admit evidence

Source intake belongs to the backend. Start with the current supplied local Zoom transcript admission. Add Fathom, Slack, founder notes, and other connectors as separately verified work. Provider webhooks or polling fetch transcript content and actual metadata through provider-authorized credentials. ChatGPT should not continuously collect source material or control provider credentials. [S1]

Store provider account, recording identity, original bytes/hash, original text/hash, version, available meeting time and timezone, ingestion time, participant/speaker labels, attribution confidence, and source-use restrictions. Unknown meeting dates remain unknown; folder labels and processing dates are not evidence of recording time. Provider summaries are useful metadata but cannot replace original dialogue.

Canonicalize provider identity before hashing for deduplication. Group duplicate Zoom/Fathom captures as the same recording so repetition is not counted as independent recurrence. Preserve changed captures as new source versions, with a reason and mapping to the original. Quarantine malformed text, impossible passage ranges, missing processing authority, or uncertain identity requiring clarification.

Assign stable passage IDs and explicit original-text offsets. The v1 adapter uses zero-based Unicode code-point offsets and half-open intervals [start,end); Python and JavaScript consumers must convert consistently. Retain original byte digests separately. Optional start/end seconds describe actual available timing and must not be invented from text offsets.

Weekly manifest

Configure WIA's business timezone, seven-day source window, admission cutoff, late-arrival policy, and review day. The client's Asia/Kolkata timezone is known for this document and experiment; it does not establish WIA's business timezone. Saturday 04:00 is a selected proposal awaiting this operating decision.

A frozen manifest contains batch/week/run identity, start/end dates, cutoff, timezone, source IDs/versions/hashes, passage counts, omissions, and policy version. Only the backend changes readiness from PREPARING to READY. A complete manifest requires every admitted source to be accessible and authorized, not merely a count of files.

Late sources do not silently alter an executing manifest. Proposed policy: create a supplemental batch or a new explicit run against an expanded manifest. Preserve the original run and review lineage. A source correction or revoked permission invalidates dependent evidence and requires reassessment. An empty eligible week is a truthful NO_SOURCES result, not an invented opportunity list.

Whole week coverage policy

The agenda's current contract supplies lossless whole-week input and rejects requests that would omit, summarize, or silently split dialogue. Preserve that contract for the initial pilot. Preflight actual mandatory packet size and the observed executor's usable capacity. A failure to fit becomes CONTEXT_LIMIT with an operator-visible next action. [S1]

The earlier selected architecture proposed source-by-source extraction for larger weeks. That would change the current editorial contract. A future approved coverage_policy_version may authorize passage partitions, coverage ledgers, and final whole-week synthesis. Validate that method on overlapping context and unseen sources before activation. Never label partial extraction as complete

merely because each requested segment returned.

The future ledger should record each assigned source range, retrieved digest, accepted extraction batch, exclusions, and failed range. Coverage percent, if shown, measures accepted assigned ranges, not insight completeness. A fully read transcript can still contain missed opportunities; editorial recall needs separate evaluation.

9 Moments angles and Gate 1

Stage 1 discover and admit moments

The weekly task probes access, discovers an eligible extraction intent, atomically claims it, and retrieves its exact manifest, evidence, strategy, and preference snapshot. The extraction skill identifies useful exchanges with setup, reasoning, client contribution, WIA contribution, qualifications, uncertainty, and sensitivity. It returns candidate moments with exact evidence anchors.

The backend first stores the raw result artifact, then validates source membership, source version, anchor ranges, quotations, attribution, and supported claim status. Admission creates key_moments only for valid records. Failed candidates remain auditable with actionable findings. The job ends as completed only under its configured coverage contract; partial accepted output produces a visible partial state and does not authorize an unqualified complete-week claim.

Moments are evidence-backed observations, not headlines. Preserve distinctions among advice, a client report, a proposed process, implemented work, and verified outcome. A model confidence number cannot resolve an unknown speaker. Material attribution uncertainty requires original-source verification or anonymized treatment.

Stage 2 develop and compare opportunities

Once extraction prerequisites pass, the backend queues ANGLE_GENERATION with validated moments and whole-week scope. The angle skill creates substantive central points, intended audience, reader takeaway, primary pillar, strategic purpose, selection reason, source demonstration, WIA contribution, development direction, claim boundaries, and research gaps. It may combine moments while retaining precise lineage.

Apply SELECT, REFRAME, HOLD, INTERNAL, and SKIP editorial dispositions from the handbook as a proposed migrated contract. A useful idea missing a factual segment should remain HOLD with the exact missing requirement. Non-public objections may become INTERNAL intelligence. Avoid padding to a count; an empty qualified set is valid. Reconcile these fields with the active production contract before switching profiles. [S1]

Existing backend comparison computes angle embeddings and examines historical representatives and current-batch central points. Preserve exact numerical comparisons and history scope. Model-generated similarities are not replacements for these calculations. Similarity cutoffs remain founder-calibrated settings; the earlier pilot-specific 0.90 is not a permanent global policy. Same-source reruns and superseding families require the existing self-match rules. [S4]

If a targeted refinement can recover a distinct supported direction, allow the configured single comparison refinement pass. Store before/after reasoning and compare again. Incomplete comparisons remain unknown and may block approval under policy. Keep valid suppressed candidates in SQL; visibility is a review query, not deletion.

Gate 1 choose the direction

Whitney or an explicitly delegated approver sees title, central point, audience, strategic purpose, selection reason, disposition, and next action first. Expand evidence, source context, comparison

rationale, gaps, and claim restrictions on demand. Show whether coverage and checks are complete before enabling approval.

Actions are APPROVE, REQUEST_REVISION, REJECT, and DEFER. Source or steering questions may be answered without approving. Store actor identity from trusted authentication, reviewed entity row/hash, decision time, exact feedback, warning acknowledgements, chosen CTA where needed, and policy version. Never accept requested_by as proof of authority.

Approval and initial brief intent should commit atomically only when briefing configuration, reviewed brand bundle, current evidence, and comparison policy satisfy eligibility. If configured approval-only mode is used, label it explicitly and do not imply a brief was queued. The existing live brief workflow already binds approval and intent when enabled; the adapter should reuse that logic. [S4]

An approved angle authorizes research and brief development for that exact version. It grants neither public quotation permission nor pack approval. Rejecting a direction closes its generation intent but preserves its evidence and decision history. Deferral creates no downstream work until an explicit later decision.

10 Revision requests and concurrent editing

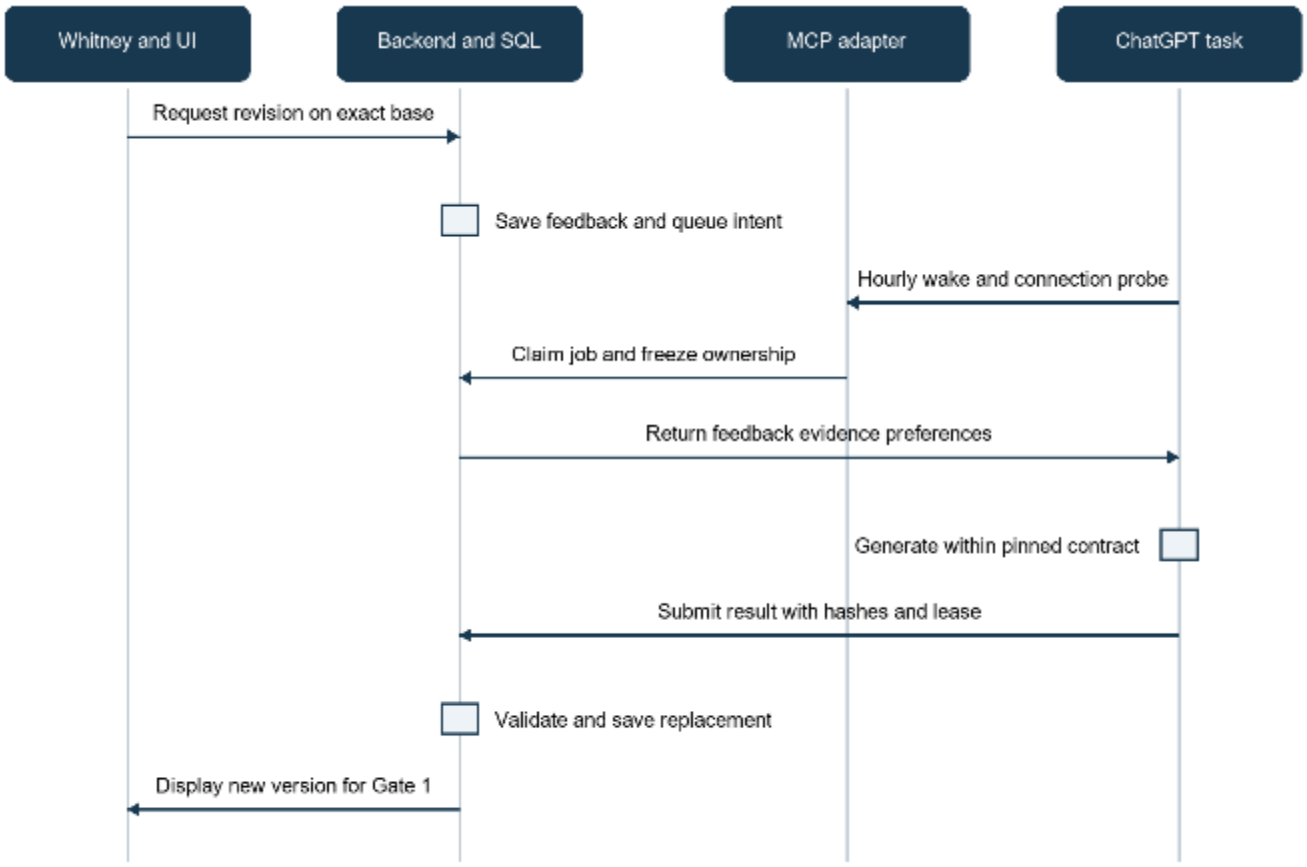


Figure 2 Angle revision sequence

Stage 3 request and process an angle revision

The browser submits entity identity, expected content hash, base revision, exact feedback, change scope, and an idempotency key to the protected backend. The backend validates identity and current version, saves feedback, creates ANGLE_REFINEMENT, and shows Revision queued in one

transaction. The original reviewed content remains accessible.

During its next eligible run, the editorial worker claims the intent and receives the base angle, exact feedback, source moments/passages, applicable policy, CTA, prior versions, unresolved requests, and relevant history. The refinement skill changes only the requested scope, explains the change, identifies unresolved evidence gaps, and preserves original support. Feedback must not become new factual proof.

The result is stored as a saved replacement linked by supersedes_id after validation. Recompute any affected comparison, embedding, and review checks. Show the previous and revised central points with a concise change summary and preserved feedback. Gate 1 reviews the new saved version.

Direct human edits

Human edits also create versioned saved content under the compact schema's protection rules. A draft not yet consumed may use the existing permitted edit behavior, but material review requests should identify a stable base. Approved or consumed records remain immutable. Any approval-relevant edit invalidates approval and applicable validation, even if it was made by an approver.

Concurrent requests

Use expected_content_hash or If-Match for every edit, revision request, and decision. Two actions on the same base cannot silently overwrite each other. If Whitney edits while ChatGPT works, the backend rejects the stale commit with STALE_INPUT, preserves the generated artifact, and offers a fresh request against the new version.

One active refinement intent per logical entity/base hash is the proposed default. A second independent comment can either join the request before claim through an explicit request supersession, or create a queued request against the later result. Do not mutate a claimed feedback packet. Keep unresolved requests inspectable and require explicit consolidation when comments conflict.

Cancellation clears future eligibility and invalidates the lease's commit authority. A tool request already executing may finish, but its result cannot become current without the commit guard. A host task stopping is distinct from backend cancellation; reconcile both in operations.

Downstream invalidation

If an approved angle is materially reframed, briefs tied to the previous hash become stale for new use. Briefs preserve their original lineage; they are not rewritten in place. Asset and pack dependency checks follow the same rule. Block new handoffs and publication of obsolete content under current policy. Previously published content remains a historical fact and can require a separate correction decision.

The backend calculates the impact graph and shows affected briefs, assets, packs, and planned calendar items. The reviewer chooses regeneration, explicit reuse under a newly reviewed dependency, or cancellation. A reuse acknowledgement cannot waive missing evidence or public-use clearance.

11 Research dossiers briefs and Gate 2

Stage 4 research an approved angle

The backend verifies Gate 1 actor authority, saved row, approval hash, current source clearance, and strategy/brand readiness before queuing research. The context packet includes every original supporting moment and qualification passage. Supplemental research cannot remove inherited

grounding through ranking.

The research workflow plans scoped questions, uses allowed transcript semantic/full-text retrieval, expands chunks to original passages, grades relevance/support, and examines prior angles and actual content use. Record query intent, permitted corpus, retrieval versions, inclusion decisions, budget omissions, and unresolved gaps. The existing implementation's hybrid retrieval, frozen dossiers, and bounded rounds should remain backend capabilities. [S4]

ChatGPT can reason over retrieved evidence and propose queries; deterministic retrieval and source access stay in backend tools. Explicitly pin corpus cutoff and run clock, because historical novelty during angle discovery can have a different scope from briefing's prior-use review. Later generated arguments may inform a current brief without retroactively changing an earlier discovery score.

Optional public research adds external framing only through an approved adapter and topic packet. The agenda reports no configured production web adapter. Do not assume scheduling gives deep research, browsing, or a licensed data source. A proposal can enable a bounded research tool after access and evidence tests. Record actual URL, title, retrieval date, source scope, and factual excerpt; external material cannot establish internal results. [S1, S4]

Freeze a dossier with evidence refs, permitted claims/statuses, prior-content map, questions, research gaps, source-use permissions, policy versions, and a digest. Empty supplemental results after completed search differ from unperformed research. An unresolved gap can permit a narrower honest draft or block development, depending on its significance.

Stage 5 write and validate a brief

The writer receives the approved angle, frozen dossier, reader promise, current strategy/offers, voice/preference snapshot, CTA steering, and exact brief contract. It produces core insight, message spine, narrative, claims, differentiation, voice direction, CTA, format treatments, creative/accessibility notes, open questions, and sources. Format direction in a brief does not create approved final assets.

Backend validators check required fields, local reference namespaces, exact quotes/ranges, claim-status tokens, qualification wording, numbers and units, current upstream hashes, and required questions. A scoped semantic review assesses source support, differentiation, and voice with cited reasons. It is advisory evidence for human review, not a new approval authority.

Preserve the existing limit of one automatic brief repair. The repair receives only actionable findings plus required grounding and contract. Remaining failures become a blocked or review-required state. Classify validator defects separately from actual content defects; the existing pilots include false quantity flags on citation labels and invalid reference/status fields. A model should not keep rewriting truthful content to satisfy a broken checker. [S4]

Generation completion means a saved draft and current checks exist. It does not mean the brief is accepted. In-flight model output that has not committed should appear as Processing, not Ready.

Gate 2 approve the development plan

The reviewer sees the brief, governing angle, selected evidence and limitations, differentiation map, permission records, unresolved questions, and checks against the exact hash. Answers can supply editorial direction or verify a destination; a human answer claiming a result requires the appropriate underlying evidence.

Actions are APPROVE, REQUEST_REVISION, REJECT, and DEFER. Any substantive edit or answered steering question triggers applicable revalidation. Required permissions, factual questions, missing lineage, and unresolved blocking findings prevent approval. Non-blocking warnings require explicit authenticated acknowledgement where policy calls for it.

A Gate 2 approval authorizes pack development within an agreed plan for the reviewed brief versions. The current backend does not automatically create a production job after this gate. Add that transition only in the proposed pack implementation, behind its own readiness switch. [S1, S4]

Stage 6 refine a brief

BRIEF_REFINEMENT consumes the saved brief/hash, feedback, exact approved angle, dossier, questions/answers, permissions, prior revisions, and selected policy versions. Changes to the core direction may require returning to Gate 1. New evidence triggers a new dossier version; it cannot silently modify an already reviewed dossier. The replacement brief returns to Gate 2 with fresh checks and an explanation of unresolved requests.

12 Pack planning content assets and Gate 3

Stage 7 agree the production plan

Before generating, the backend records a reviewed plan containing week, selected approved brief refs/hashes, ordered slots, format, channel, intended audience/purpose, owner, constraints, caption requirements, and production deadlines. One pack may draw from several approved ideas. An uncapped candidate pool does not imply a pack per candidate.

Use the agenda's defaults only as initial choices: one email, one seven to ten slide carousel, one to three reels, and long/short captions. Existing brief writer contracts have fixed eight-slide/three-reel directions in their profile; the proposed pack plan must reconcile those instructions with actual selected quantities. Do not pretend variable pack generation already exists. [S1, S4]

Stage 8 generate and validate assets

The backend creates selected content_assets rows as planned slots, with null copy/hash until generation succeeds. PACK_ASSET_GENERATION or the bounded PACK_GENERATION intent consumes the plan and approved brief hashes. Different formats should serve distinct treatments of the governing message, with evidence and claim boundaries preserved.

Email content includes subject, optional preheader, body, CTA, and destination. A carousel includes ordered slide copy, design/visual guidance, and caption variants. A reel includes script sections, estimated duration if measured or labelled estimated, shot/delivery notes, and captions. Required accessibility text should describe actual planned or final visuals honestly.

Validate every assigned slot, format limits, exact evidence/claims, voice, CTA destination, upstream brief hash, and production constraints. Store per-asset findings. A pack manifest hash covers slot order, asset IDs/copy hashes, governing brief IDs/hashes, and plan revision. Any assigned missing asset blocks pack completion. Optional slots are removed through a reviewed plan revision, not silently skipped.

Research, drafting, design rendering, and video production are separate activities. The proposed ChatGPT draft executor does not establish Canva/Descript automation or uploaded media availability. Giselle remains responsible for the agreed downstream production work.

Gate 3 approve the handoff

The approver reviews complete assets against the governing briefs, current checks, supported claims, public-use permissions, CTA, format quality, and production notes. The backend verifies exact manifest and asset hashes in the same transaction as the decision. No generation tool can create that decision.

Actions are APPROVE_HANDOFF, REQUEST_REVISION, REJECT, and DEFER. Approval authorizes the specifically agreed production handoff. It does not automatically publish, schedule provider actions, or permit unreviewed design/script changes. If the team wants one approval to cover a later delivery scope, that scope must be explicitly named and separately implemented.

Stage 9 refine selected pack assets

PACK_REFINEMENT receives target asset IDs/hashes, pack manifest, exact feedback, plan, governing briefs, evidence, and current policies. Preserve unaffected approved assets. Because the compact design gives an asset one owning pack, create replacement pack/asset records when a consumed pack changes. Regenerate only affected slots and recompute all manifest-level checks. The new pack returns to Gate 3.

13 Production handoff publication and learning

Stage 10 human production

Content Atelier exports the approved handoff with asset copy, evidence boundaries, design/shot notes, captions, CTA destinations, owners, and version identifiers. Giselle creates or edits media in the team's chosen tools. Store exported artifact digests and handoff acknowledgement. A changed visual claim, promise, or script requires renewed applicable review.

Stage 11 confirmed use

Planning and scheduling screens do not prove external publication. Record planned, scheduled, delivered, published, failed, and cancelled separately. A future publishing connector pins the exact asset hash, destination/account, authorized actor, planned time, provider idempotency key, and provider receipt. Publication becomes confirmed only after an appropriate provider response or verified human record.

Use a separate delivery worker and explicit action scope. Draft-generation MCP credentials should not have publication permissions. A lost provider response requires reconciliation by provider ID or idempotency key before resending; otherwise duplicate posts may result. Never infer a booking, sale, or engagement result from a publication receipt.

Stage 12 close the learning loop

Store approved direction, approved wording, handed-off version, actual published copy, date/channel, and outcomes as distinct facts. Associate outcome units, timeframe, source, and attribution limitations. A high engagement count does not establish causal commercial impact or a new founder preference.

Review recurring feedback for scoped reusable candidates and compare later unseen-source performance. Maintain holds and internal insights with their exact missing prerequisites or non-public destination. Resurface a hold only when the recorded gap is resolved or a reviewer explicitly narrows the claim.

14 Approval semantics and state separation

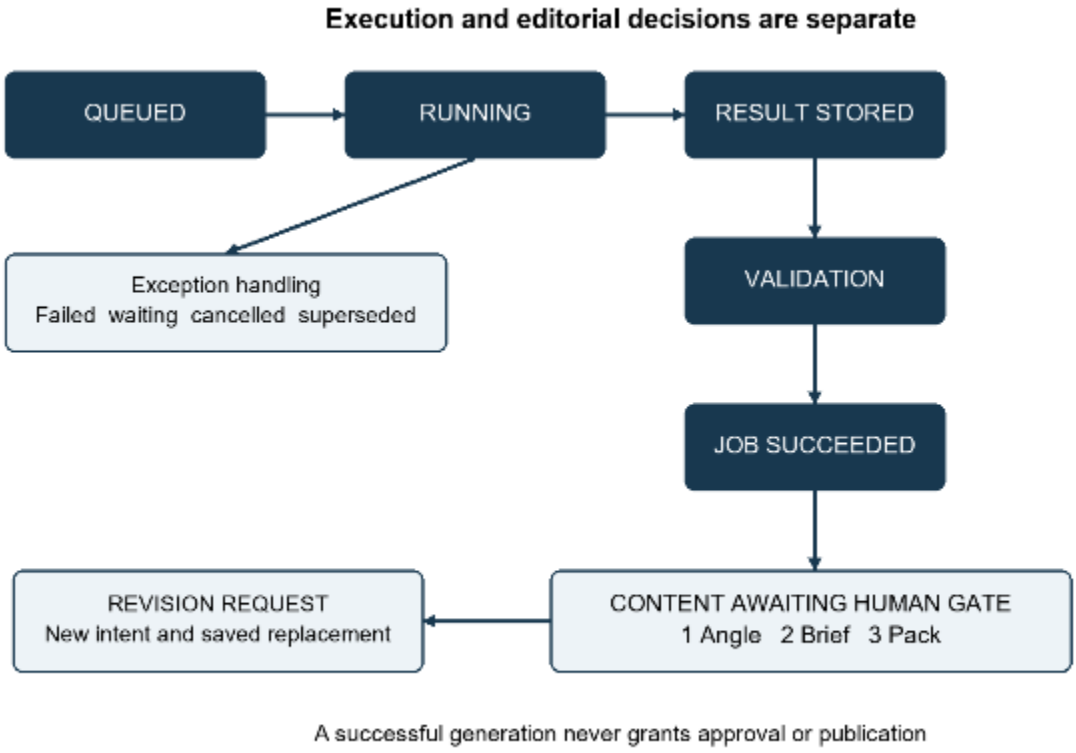


Figure 3 Jobs and review states

Generation jobs, content review, overall runs, and publication need distinct state machines. A completed generation job may produce an angle awaiting Gate 1. A run can wait for a human while no job is active. A published item remains published even when a newer revision awaits review.

Object	Proposed external states
Source batch	PREPARING READY PROCESSING COMPLETE PARTIAL BLOCKED SUPERSEDED
Job	QUEUED RUNNING WAITING_INPUT SUCCEEDED FAILED CANCELLED SUPERSEDED
Content review	DRAFT CHECKING AWAITING_REVIEW REVISION_QUEUED APPROVED REJECTED DEFERRED STALE
Run	QUEUED RUNNING WAITING_REVIEW PARTIAL COMPLETED FAILED CANCELLED
Publication	PLANNED SCHEDULED DELIVERED PUBLISHED FAILED CANCELLED

These are v1 adapter states, not a declaration of current SQL enum values. Map SUCCEEDED to the current job completed value, RUNNING to running, QUEUED to queued, and the existing run waiting_for_review where appropriate. Migrate additional states deliberately or represent their reason in

typed progress/result fields until the schema changes. Do not write invented values into constrained columns.

Readiness requires current validation and prerequisite decisions, not a model-assigned status. The model cannot supply approved_by, approval timestamps, visibility, authoritative similarity, or published status. Backend-generated metadata travels separately from generated editorial content.

Approvals bind actor, capability, tenant, gate, saved entity row, content or manifest hash, policy version, time, and acknowledgement references. Approval repeats with the same idempotency key return the original decision; incompatible repeats fail. Revalidation locks the entity/dependencies and verifies all relevant hashes before commit.

For an edit after approval, preserve the original decision on its original row. Clear eligibility on the replacement and propagate stale dependency status. A rejection or revoked source restriction may require an impact review but never erases the history of earlier decisions. Separate internal processing permission, public naming/quotation clearance, editorial direction approval, and external delivery authorization throughout.

15 Application events and transactional commands

Exact event envelope

The following is the proposed Content Atelier domain event contract v1. It is separate from the MCP Events transport envelope. Field names, required fields, and types are normative for the proposed adapter. All object contracts reject unknown keys unless a specific typed extension is listed. UTC timestamp strings use RFC 3339 with a Z suffix. IDs are opaque server-resolved identifiers; examples are fictional.

```
{
  "schema_version": "atelier.event.v1",
  "event_id": "evt_01",
  "tenant_id": "wia",
  "event_type": "revision.requested",
  "occurred_at": "2026-10-14T07:00:00Z",
  "actor": {"kind": "human", "id": "user_01"},
  "aggregate": {
    "kind": "angle", "id": "angle_301",
    "version": 1, "content_hash": "sha256:64-lowercase-hex"
  },
  "run_id": "run_01",
  "job_id": "job_501",
  "correlation_id": "corr_01",
  "causation_id": null,
  "idempotency_key": "revision-click-01",
  "payload": {
    "feedback_id": "fb_021", "request_type": "REFRAME",
    "feedback_artifact_id": "artifact_feedback_021"
  }
}
```

Every envelope field is required; run_id, job_id, and causation_id may be null when not applicable. actor.kind is human, service, or executor; the server resolves actor.id. aggregate.kind is batch, moment, angle, brief, pack, asset, preference, job, or publication. version is a positive integer or null for a non-versioned aggregate; content_hash is a valid digest or null before content exists. event_type selects the exact payload below. The placeholder digest in this illustration is not a valid real digest.

Event type	Required payload fields	Consequence
source.batch_ready	batch_id manifest_hash coverage_policy_version	Queue extraction if enabled and no existing intent
revision.requested	feedback_id request_type feedback_artifact_id	Queue refinement on pinned base
gate.decided	gate decision reviewed_hash acknowledgement_ids next_intent_id	Persist exact decision; queue eligible next intent if configured
job.claimed	attempt lease_expires_at executor_kind	Record ownership milestone without token
job.result_admitted	result_type artifact_id admitted_refs validation_hash coverage_status	Commit saved content and job outcome
job.issue_reported	issue_code retryable issue_artifact_id	Pause or fail with explicit next action

Event type	Required payload fields	Consequence
job.superseded	superseding_intent_id reason	Invalidate old commit eligibility
preference.decided	preference_version decision snapshot_hash rule_ids	Activate revoke or roll back reviewed snapshot
handoff.confirmed	manifest_hash export_artifact_ids recipient_actor_id	Record actual authorized production handoff
publication.confirmed	asset_id asset_hash channel provider_receipt_id published_at	Record confirmed use only

Typed constraints: gate is 1, 2, or 3; decision is one of the gate's actions; request_type is COPY_EDIT, REFRAME, EVIDENCE_FIX, CTA_CHANGE, or FORMAT_CHANGE. acknowledgement_ids and artifact ID arrays contain unique opaque IDs. next_intent_id and superseding_intent_id may be null when no replacement or successor exists. coverage_status is COMPLETE, PARTIAL, or NOT_APPLICABLE. Digest fields use sha256 plus 64 lowercase hex digits. Narrative reason strings are 1 to 2000 characters; feedback text is held in its protected artifact rather than broadly distributed events.

Frontend command contract

Proposed POST /api/v1/{angles|briefs|packs}/{id}/revision-requests requires an authenticated editor, an Idempotency-Key header, and body fields base_revision (positive integer), expected_content_hash (digest), feedback (1 to 8000 non-whitespace characters), request_type (enum above), and target_asset_ids (unique IDs; empty except a scoped pack revision). The server resolves tenant and actor. The response contains request_id, feedback_id, job_id, entity_id, base_revision, status REVISION_QUEUED, and server-created created_at.

Proposed POST /api/v1/{angles|briefs|packs}/{id}/decisions requires approver capability, Idempotency-Key, and body fields expected_revision, expected_content_hash, decision, comment (nullable string up to 8000), acknowledgement_ids, and selected_cta (nullable typed CTA). The backend returns decision_id, gate, reviewed_hash, decision, recorded_at, and next_job_id (nullable). Require selected_cta only where current policy needs it; it is not proof of offer scope.

For both commands, a matching retry returns the original response. The same key with a different canonical body returns IDEMPOTENCY_CONFLICT. An obsolete hash returns STALE_INPUT. Unauthorized actors return 403; unavailable identity 401; invalid JSON/schema 400 or 422 according to the API's documented convention. Persist the change, feedback/decision, successor intent, and milestone event together or roll them all back.

Events and delivery

Keep business milestones in pipeline_events. Browser subscriptions read committed tenant-scoped events with sequence cursors. Heartbeats update jobs rather than creating an event each time. Never put raw transcripts, credentials, or large model output in event details.

For external notification delivery, add an outbox status to a separate outbox store or a deliberately extended event design. Write the business event and outbox reference in the same transaction. Dispatch at least once; consumers deduplicate event_id. Preserve failed delivery attempts and replay cursors. Internal SQL event order and an external webhook's arrival order can differ, so consumers reload current state rather than trusting event ordering alone.

16 Job contracts ownership and result admission

Exact job record

The backend returns the proposed job envelope below after successful claim. Every listed key is required; nullable keys are explicit. The model cannot create or mutate this record directly.

```
schema_version: literal atelier.job.v1
job_id, run_id, tenant_id, week_id: ID
job_type: JobType
target: {kind: batch|angle|brief|pack|asset, id: ID, revision: int|null}
base_content_hash: Hash|null
feedback_ids: unique ID[]
dependency_refs: RecordRef[]
context_id: ID
context_hash: Hash
input_hash: Hash
executor_kind: literal chatgpt_plus
required_skill: {name: ID, version: ID, digest: Hash}
policy_versions: {strategy: ID, brand: ID, offers: ID,
                  preferences: int, validators: ID, coverage: ID}
priority: integer 0..100
status: literal RUNNING
attempt: integer 1..3
created_at, not_before, deadline_at: UTC
lease_token: opaque secret string
lease_expires_at: UTC
limits: {max_result_bytes: integer, max_tool_calls: integer,
         max_repair_passes: integer 0..1, max_runtime_seconds: integer}
```

RecordRef is {kind, id, revision, content_hash}; kind names an allowed source or content type, revision is positive integer, and the digest is mandatory. For original source refs, revision is the preserved source version. For a batch target revision can be null, but its manifest digest appears in base_content_hash. For a new downstream generation target, base_content_hash may be null and dependency_refs pins the approved input. input_hash covers immutable intent, context, dependencies, and policies, excluding volatile lease/attempt fields.

Canonical hashing and compatibility

Use the existing server canonical_hash implementation for business hashes: serialize JSON with sorted object keys, ensure_ascii false, and comma/colon separators, encode UTF-8, then compute SHA-256. Arrays retain their order, strings retain their exact Unicode, and null remains explicit. The v1 wire format prefixes the existing lowercase hexadecimal digest with sha256 without changing its underlying value. The backend computes hashes; clients echo returned values rather than independently guessing serialization. [S14]

Reject duplicate JSON keys and non-finite numbers at ingress. Version the hash projection separately for each content type so excluded/generated metadata is unambiguous. Context hashing excludes its returned digest; result hashing excludes credentials and caller idempotency key; pack hashing includes ordered selected slots and their dependency hashes. Do not normalize original transcript text to obtain a more convenient quotation match.

Proposed initial budgets are one job per wake, one automatic repair where permitted, at most three delivery/execution attempts requiring appropriate retry authority, a 30 minute maximum lease lifetime, and a 256 KiB generated-result body. These are application pilot defaults, not measured ChatGPT limits. Preflight whole-week input separately. Tune only with measured durations and content requirements.

JobType	Prerequisites	Accepted result type
MOMENT_EXTRACTION	READY frozen batch and processing authority	MOMENTS
ANGLE_GENERATION	Valid extraction and required coverage	ANGLES
ANGLE_REFINEMENT	Saved base angle and immutable feedback	ANGLE_REVISION
BRIEF_RESEARCH	Exact Gate 1 and reviewed context	DOSSIER
BRIEF_GENERATION	Approved angle and frozen valid dossier	BRIEF
BRIEF_REFINEMENT	Current approved direction and saved brief feedback	BRIEF_REVISION
PACK_GENERATION	Gate 2 briefs and reviewed slot plan	PACK_ASSETS
PACK_ASSET_GENERATION	One selected slot and approved brief	ASSET
PACK_REFINEMENT	Saved manifest selected targets and feedback	PACK_REVISION
PREFERENCE_PROPOSAL	Resolved feedback and reviewed source decisions	PREFERENCE_CANDIDATES

These external intent names map to existing stage sequences such as moment_extraction, angle_generation, angle_embedding, angle_matching, and the current brief stages. Embedding/comparison stay backend jobs. Do not claim those stages through MCP. Add intent metadata/adapter mappings through reviewed migrations; this catalogue does not assert current SQL columns or enum support.

Discovery and claim

get_next_job is a read-only eligibility hint and creates no reservation. claim_job rechecks tenant, executor, switches, dependencies, not_before, deadline, policy compatibility, active refinement conflict, and current approval hashes in a locked transaction. Use the existing row-lock/SKIP LOCKED pattern. Return NO_WORK if another executor claimed the hint.

A random lease token and monotonically increasing attempt identify commit authority. The token is credential-bound and cannot authorize another tenant or job. Check it on context reads, progress updates, issue reports, and result submission. Never log the token or include it in public process events.

ChatGPT cannot run a local background heartbeat thread through this cloud MCP design. Renew at bounded tool checkpoints through report_job_progress. Give the initial lease enough time for the measured longest generation interval. If an uninterrupted call exceeds it, a stale result is rejected. Do not let a backend proxy manufacture continued executor liveness. Distinguish last tool activity from

actual model thinking.

Submission envelope

submit_job_result accepts only job_id, lease_token, attempt, input_hash, base_content_hash, context_hash, idempotency_key, result_type, payload, and change_summary. All keys are required; base_content_hash and change_summary may be null. payload must match the allowed JobType/result_type pair and catalogue in Appendix B. The server computes result hash from canonical envelope content excluding lease_token and idempotency_key.

The idempotency identity is tenant plus job plus input hash plus caller key. Persist the submitted envelope and digest before admitting business output. An exact repeat returns its original receipt even after the lease has closed; first authenticate the same principal/scope. A changed envelope under that key is a conflict. A new submission must hold a live matching lease and current dependencies. Exactly-once admission is a database invariant; it is not a promise of exactly-once model execution.

Perform bounded deterministic checks synchronously. If semantic review or embedding takes longer, return VALIDATION_PENDING with a stored submission_id and receipt; the backend owns subsequent validation. get_submission_status returns progress or final admission. A received result closes generation ownership but does not pretend content is ready. Durable validation retries reuse the saved result.

Admission transaction

Lock job, dependencies, relevant content, and receipt. Recheck lease for a new submission, compare hashes, enforce source access and current revocations, validate references and result type, create saved records, update result_refs, mark generation SUCCEEDED, append milestones, and expose AWAITING_REVIEW only after required checks. If validation has blockers, preserve the artifact and typed findings without marking the content approved.

An artifact/database split failure is reconciled by submission_id and hash. Replay durable artifacts after checking compatibility; never ask the model to regenerate merely because the SQL commit failed. The current repository deliberately marks expired leases failed and requires explicit retry after artifact review. Preserve that policy for the pilot. [S14]

17 MCP tool catalogue and exact interfaces

The adapter exposes focused tools over authenticated HTTPS. Tool descriptions explain the supported operation and limitations; accurate annotations describe actual state changes. Metadata is guidance for host risk decisions, not an authorization boundary. get_next_job is read-only only because it does not reserve anything. claim_job and progress/result tools are writes even if their effects are reversible. [S13]

Common types are defined in Appendix B. Below, every input key is required unless marked optional; objects reject additional properties. JobBound means {job_id: ID, lease_token: Secret, attempt: positive integer}. Application errors have the consistent Error shape below. Read tools return only the credential's tenant and permitted corpus.

Tool	Exact input	Structured success output
check_connection	{}	status ok; api_version; principal_id; tenant_id; allowed_job_types; permission_revision; server_time; execution_enabled

Tool	Exact input	Structured success output
get_next_job	queue DISCOVERY or EDITORIAL	job_hint nullable {job_id job_type input_hash}; server_time; next_eligible_at nullable
claim_job	job_id; expected_input_hash; idempotency_key	claimed boolean; job nullable Job; receipt_id
get_job_context	JobBound; context_id	ContextPacket plus context_hash
get_source_segment	JobBound; source_id; source_version; passage_ids unique array	segments SourceSegment array; source_ref RecordRef
search_evidence	JobBound; query; corpus_id; limit 1..20; cursor nullable	hits EvidenceHit array; next_cursor nullable; coverage
get_preference_snapshot	JobBound; preference_version	version digest markdown active_rule_ids
report_job_progress	JobBound; step; completed_units nullable; total_units nullable	accepted lease_expires_at server_time
submit_job_result	SubmissionEnvelope	submission_id receipt_id result_hash status admitted_refs validation_findings duplicate
get_submission_status	job_id; submission_id	submission_id status admitted_refs validation_findings updated_at
report_job_issue	JobBound; issue_code; summary; missing_refs; retryable	receipt_id status next_action

No tool accepts arbitrary SQL, external URL fetching, actor impersonation, gate decisions, preference activation, or publication. search_evidence is available only to research-capable job types and the frozen authorized corpus. A separate future public_research tool requires a reviewed provider and packet; do not add an unbounded fetch URL argument to the pilot.

get_job_context returns required inputs and a manifest of supplementary references. Sources omitted from the inline packet must be explicitly available through permitted reads; mandatory whole-week text cannot be treated as supplied merely because it is listed. A tool delivering paginated mandatory content must record complete delivery and the model must retrieve every page under an approved coverage policy.

Success and error shapes

Return model-readable summary text alongside structuredContent matching the output schema. State clearly whether data was stored, admitted, pending validation, or rejected. A conversation reply is never the business receipt.

```
{
  "code": "STALE_INPUT",
  "message": "The reviewed angle changed while this job ran.",
  "retryable": false,
  "job_id": "job_501",
  "correlation_id": "corr_01",
```

```
  "details": [{"path": "base_content_hash", "rule": "current_hash"}],
  "next_action": "REQUEST_NEW_REVISION"
}
```

Error.code is INVALID_ARGUMENT, UNAUTHENTICATED, FORBIDDEN, NO_WORK, IDEMPOTENCY_CONFLICT, STALE_INPUT, LEASE_EXPIRED, CONTRACT_MISMATCH, VALIDATION_FAILED, CONTEXT_LIMIT, DEPENDENCY_BLOCKED, RATE_LIMITED, SERVICE_UNAVAILABLE, or INTERNAL_ERROR. message is 1 to 2000 characters and contains no secrets. job_id may be null. details is an array of {path, rule} strings. retryable is a server decision; the executor's issue report is advisory. next_action is STOP, RETRY_IDENTICAL, WAIT, REVIEW_SOURCE, RECONNECT, REQUEST_NEW_REVISION, or OPERATOR_REVIEW.

Authentication failures use transport 401 with appropriate discovery/challenge metadata. Schema/protocol errors follow the negotiated MCP version. Valid tool execution with an application rejection uses isError true and the typed safe error content; never disguise a rejected write as success. Distinguish JSON-RPC errors from domain errors in logs.

Plugin and skill structure

Package orchestration, extraction, angles, angle refinement, research, briefs, pack drafting, pack refinement, and preference proposal skills under one versioned plugin identity. Each skill declares its triggers, permitted JobTypes, mandatory reads, output contract, evidence rules, stop conditions, and bounded repair policy. Use static approved examples to illustrate judgement.

The task explicitly names the orchestration skill rather than relying solely on automatic matching. The worker validates required_skill and API compatibility before using tools. Store reported skill version for traceability but recognize that self-report is not execution attestation. Contract checks and outcome evaluation remain necessary.

Procedure releases receive a changelog, digest, offline fixtures, account-specific manual/scheduled smoke tests, and a rollback path. Refresh/reinstall or rescan as the relevant plugin deployment requires. Do not overwrite an installed client preference file and assume future cloud tasks received it.

18 Validation and review policy

Layers of validation

Validate inbound HTTP size, content type, JSON shape, types, enum values, Unicode/NUL constraints, string lengths, and unexpected fields before invoking domain operations. The experiment's 64 KiB body limit was a tiny receipt limit; production transcripts and draft packs need deliberately chosen route-specific limits. Proposed pilot control routes use 64 KiB; results use 256 KiB; mandatory source-context reads are paginated or size-checked under their approved coverage policy. [S2]

Layer	Checks	Blocking behavior
Identity and eligibility	Tenant scope actor capability switches job executor dependencies	Reject without side effects
Contract	Required fields known keys sizes enums stage/result compatibility	Preserve rejected artifact and findings
Referential integrity	Existing authorized IDs source versions passage ranges family/slot links	Reject unknown or cross-tenant references

Layer	Checks	Blocking behavior
Evidence fidelity	Exact quote text attribution qualifiers claim status numbers units timeframe	Block unsupported material claims
Current state	Base input context policy approval and manifest hashes live lease	Reject stale admission
Editorial checks	Distinct central point audience purpose WIA contribution voice CTA	Human review and configured semantic findings
Permissions	Internal processing naming quotation public-result clearance	Block use exceeding permission scope
Coverage	Frozen manifest all mandatory sources gaps omission reasons	Display PARTIAL or block configured next stage

Every finding carries finding_id, rule_id, severity BLOCKER/WARNING/INFO, field path, referenced evidence IDs, description, suggested action, validator version, assessed content_hash, and source NORMALIZER/DETERMINISTIC/SEMANTIC/HUMAN. Unknown or unavailable checks are NOT_RUN, never PASS. Human acknowledgement binds exact finding and content hash; changed content needs reassessment.

Evidence contract

Resolve quotations against original text, not chunk_text or a model paraphrase. Allow a documented quote-normalization policy only if it is versioned and cannot conceal omitted qualifications. Separate quotation, supported proposition, and editorial inference. Exact text match proves extraction fidelity, not that a proposition follows from the passage.

Use explicit claim statuses: PROPOSED, RECOMMENDED, IMPLEMENTED, REPORTED_OUTCOME, VERIFIED_OUTCOME, and EDITORIAL_INFERENCE. Migrate the active source-token vocabulary through an adapter rather than replacing existing strict tokens silently. A speaker reporting a result is not independent verification. Keep inquiries, consultations, bookings, revenue, and ad-attributed revenue separate, including approximate wording and timeframe.

Semantic and human review

The scoped semantic reviewer assesses support, meaningful differentiation, audience breadth, and voice using a supplied dossier and explicit rubric. Request concise reasons and references; do not require hidden chain-of-thought. It cannot waive deterministic failures or approve a gate. A second pass is useful only for an unresolved material concern, not as an unlimited polish loop.

Whitney assesses whether the direction matters, whether it reflects WIA's thinking, and whether she would publish its governing message. Giselle assesses production completeness and practical format constraints. Engineering assesses schema, state, retrieval, reliability, and validator defects. Preserve these different decisions so editorial disagreement is not recorded as a transport failure.

Cache and replay

A cache key covers canonical inputs, source versions, approved dependency hashes, strategy/brand/offers/preference snapshots, prompt/skill/contract versions, generation settings where known, and relevant retrieval configuration. Replay requires current clearance and unchanged dependencies. Cache reuse keeps original generation age and provenance and shows no new provider

dispatch.

Scheduled ChatGPT may not expose token usage, exact internal prompts, or itemized model cost. Mark those fields UNKNOWN. Never attribute an existing OpenRouter pilot's eight-call orchestration or cost to a single scheduled ChatGPT run. Preserve model output and actual MCP tool receipts that the backend can observe.

19 Optional custom MCP event execution

Separate pilot track

Current OpenAI developer documentation provides a custom event route beyond polling. It describes events/list, events/subscribe, and events/unsubscribe on an authenticated MCP 2.0 endpoint, webhook delivery, signed callback verification, and subscription persistence. It excludes polling/streaming event delivery in that integration. The receipt experiment's legacy protocol check did not test this route. [S11]

Proposed event name atelier.job_ready has subscription arguments {queue: DISCOVERY|EDITORIAL}. Its application payload is {event_id: ID, job_id: ID, job_type: JobType, input_hash: Hash, queue: Queue}. Tenant filters come from authentication. Events announce availability; the model still calls claim_job and gets current context. No transcript or feedback text appears in the notification.

Wrap that payload in the official event transport, with eventId, name, timestamp, data, and cursor. Implement callback verification, expiry, signatures, retry handling, and security exactly against the current guide rather than inventing a generic webhook endpoint. The official method names and transport fields above are an integration target; account availability and runtime behavior require validation. [S11]

Proposed server controls include encrypted subscription secrets, authorization-scoped filters, deterministic subscription identity, DNS/IP validation before each callback connection, redirect refusal, standard signature verification, replay protection, and deduplication. Never allow a supplied callback to target private/internal services. Persist retries and stop/reconnect reasons.

Prevent feedback loops

Emit job_ready only when an eligible intent newly becomes executable. claim, progress, and result admission events do not create another event for the same intent. A successor job gets its own event only after prerequisite validation. Use stable event IDs across delivery retries and suppress duplicate wakes. The same job may be noticed by polling and an event task; the database claim guard decides ownership.

Keep hourly polling as a recovery path after the event pilot passes. It can find a missed notification, expired subscription, or retryable queued job. Callback acceptance does not prove a job ran; backend result admission remains completion evidence. Do not promise immediate latency or exact delivery under events until measured.

Event acceptance tests

Verify the actual Plus account can discover the event, authorize the subscription, verify the callback, receive a backend-emitted event with the tab closed, claim the job, and store a valid result. Test duplicate notification, concurrent hourly wake, expired subscription, secret rotation, revoked access, endpoint outage, and a generated update that could create a loop. Only then compare latency and maintenance cost against polling.

20 Security privacy and permissions

Connected identity and authorization

Authenticate private reads and writes. OpenAI's plugin guidance specifies OAuth 2.1 and MCP authorization metadata for authenticated integrations, and server-side scope enforcement for every tool call. The current receipt test is not evidence of production OAuth. [S12]

Use the existing Content Atelier identity system to map the connected MCP principal to a tenant-scoped execution capability. Proposed scopes are atelier:jobs:read, atelier:jobs:claim, atelier:context:read, atelier:results:write, and atelier:issues:write. Grant only configured job families and corpus access. Separate editorial approver capabilities and publisher permissions from these scopes.

Validate issuer, audience/resource, signature, expiry, scopes, and tenant association on every request. Use PKCE and current discovery/client registration practices supported by the integration. Encrypt refresh tokens and signing secrets at rest. Revoke tokens on disconnect, departure, compromised account, or scope change. Never treat Host/Origin checks as authentication or a tunnel URL as a secret.

Approval layers

ChatGPT app permissions decide whether the host will execute a tool. Backend capabilities decide whether the operation is allowed. Editorial gates decide whether content is acceptable. A configured host permission cannot override backend role checks; a Gate 1 approval cannot clear a host prompt.

DOCUMENTED: Personal app settings may allow reads, low-risk actions, or supported actions without another prompt, depending on account/app/connection policy. Some actions may be denied or require approval. Availability varies, and the broadest option is not a standard account-wide selector. [S15]

Use narrowly scoped reversible draft writes for the pilot. Record actual account permission settings and retest after schema/tool changes. If a host pause occurs, show Attention needed in Content Atelier through observed backend inactivity/issue state; an operator may need ChatGPT to resolve it. Do not relabel a write read-only to avoid approval. High-risk deletion, publication, sharing, or money movement remains outside the generation plugin.

Data handling

Only send the source material and business context needed for the job, with permitted redaction/anonymization where editorial meaning survives. Maintain an inventory of data transferred to ChatGPT and connected providers. A personal Plus account is not an organizational compliance guarantee, and project-only memory is not a contractual data-residency or deletion boundary.

DOCUMENTED: ChatGPT data controls let users disable model-improvement use of eligible conversations; account/workspace policies and feature behavior still apply. Verify actual settings before processing client data. Turning off training or memory does not erase already stored chats or backend artifacts. [S16]

The same guidance says voluntarily submitted response feedback may include the associated conversation for model improvement despite opt-out. Establish a policy for confidential material before using host feedback controls; ordinary Content Atelier editorial feedback stays in its backend records. [S16]

OpenAI's security guidance recommends least privilege, input validation, audit trails, minimized data, retention policy, and PII-redacted logs. Apply those principles to Content Atelier's services and artifact store. [S17]

Define business retention windows for original transcripts, outputs, feedback, preferences, audit metadata, and backups. Deletion is coordinated across SQL, artifacts, caches, embeddings, exports, source providers, and retained ChatGPT records where supported. Revoke access immediately, then execute approved retention/deletion workflows. A pseudonymized preference example can retain editorial learning only if it does not preserve prohibited personal information.

Deployment controls

Use a stable production tunnel or directly hosted HTTPS endpoint, secret-managed infrastructure, private PostgreSQL, encrypted storage, restricted service ingress, and monitored health. Limit rate, body size, query range, tool result size, and concurrent claims. Keep administrative and receipt-debug endpoints authenticated or disabled.

Sign and version plugin releases, audit changed tool schemas, and prevent an unreviewed skill from broadening backend scope. Protect input/output exports and avoid presenting secrets or complete private transcripts in operator logs. Test tenant-ID tampering, forged actor IDs, traversal, injection, replay, cross-job lease use, and permission revocation during generation.

21 Scheduling capacity and product limitations

Two proposed schedules

Weekly Discovery wakes at the proposed Saturday 04:00 in the confirmed business timezone. It handles prepared extraction and angle intents, with deterministic prerequisites between them. Editorial Workflow Worker wakes hourly for angle revisions, research/brief jobs, brief revisions, and later pack work. Backend readiness, not the scheduler's interpretation of a button click, decides eligibility.

The backend closes the weekly source window and creates durable intents on its own. A skipped discovery wake leaves a recoverable eligible job. Later discovery wakes find unfinished approved work before creating duplicates. If a multi-stage week needs more than one wake, choose an explicit continuation policy or allow the hourly worker to claim eligible discovery continuation stages. This is a required queue policy, not something to leave to the model.

Proposed pilot queue policy: DISCOVERY can claim moment/angle generation; EDITORIAL can claim refinement and brief work plus already-created discovery continuations. Only the backend weekly coordinator creates discovery runs. One wake claims one bounded generative job, then optionally claims the next only if permitted by a measured remaining-runtime budget. Start with one per wake until durations and limits are known.

Use a timezone-aware calendar definition and store the saved task's actual schedule, timezone, plugin identity, model selection where visible, and permission configuration as an operator record. WIA's timezone and daylight-saving behavior remain OPEN. Do not infer them from the experiment's India timezone.

Durable task instructions

The following are proposed task prompts to test and install later. They contain no client transcript text or secret. Actual tool/skill names must match the released plugin.

```
Weekly Discovery
Run Content Atelier's orchestration skill for the DISCOVERY queue.
First call check_connection and confirm the authorized WIA account.
Retrieve an eligible job and claim it. Use its pinned context,
skill version, evidence rules, and result contract.
Process only the claimed intent. Retrieve all required source context.
Submit the structured result through MCP and inspect the receipt.
Report missing evidence or limits through report_job_issue.
Never approve a gate or publish. If no work exists, finish quietly.
```

```
Persist all editorial findings and issues to Content Atelier.
Notify the operator only for a meaningful failure or required action.
Editorial Workflow Worker
Run Content Atelier's orchestration skill for the EDITORIAL queue.
Probe connection, discover and atomically claim one eligible intent.
Retrieve the exact base version, feedback, evidence, current pinned
preferences and relevant history. Apply the specified stage skill.
Renew ownership at tool checkpoints within the granted limit.
Submit the result with the input and base hashes and inspect receipt.
For an uncertain write, retry the identical submission only.
Stop on a stale lease or changed dependency and record the issue.
Do not infer facts from remembered context or bypass human gates.
If no work is eligible, finish quietly. Save results in Content Atelier.
```

Latency and capacity

With successful hourly wakes and a ready job arriving uniformly within the interval, expected polling wait is about 30 minutes and nominal maximum wait nearly 60 minutes before execution starts. Add wake delay, earlier queued work, generation, validation, and repair. This is a planning calculation, not a service promise. A missed wake can make latency much longer.

Capacity depends on eligible wakes, jobs completed per wake, success rate, and stage duration. For example, one job per hourly wake gives a theoretical 24 opportunities per day before account limits or failures, not 24 guaranteed completed assets. One source week may require several generative stages. Revisions should have higher priority than new optional proposals, with age-based priority to prevent starvation.

Do not stagger many hourly tasks to circumvent account cadence limits. Account-specific active-task counts, usage limits, model access, available credits, and permitted recurring frequency need verification at setup. Current documentation differs by cloud/local/chat/task surface; do not carry an older numeric limit forward without checking. Schedule model retirement and plugin compatibility reviews.

Limitations visible in the product

Users can queue revisions in Content Atelier, but the polling design does not provide instantaneous execution. The UI should show queued since, last executor contact, processing step, and validation status. It should not display a guaranteed completion time or a fictitious LLM percentage.

This pilot does not provide a conventional backend-controlled model API, deterministic internal reasoning, exact prompt/token observability, or a proven scalable service identity. It depends on an account's connected apps, approval settings, product rollout, model availability, and schedule status. A successful one-time test does not establish ongoing service guarantees or the right to share a subscription account among team members.

If measured revision latency, throughput, observability, tenancy, or operational control fails the product requirements, keep the same backend contracts and use an explicitly authorized API-based or other supported executor for selected stages. Plus subscription execution and API billing are separate routes; budget API usage separately. Evaluate current terms and the chosen distribution/hosting model before commercialization; this architecture is not a legal determination.

22 Recovery monitoring and operational ownership

Failure matrix

Condition	Backend behavior	Recovery and visible state
No eligible work	Read-only NO_WORK; no model result intent	Quiet task completion; last probe visible
Missed wake or paused task	Durable queue remains; backend watchdog detects age	Operator checks connection schedule and pending approval
Token expired or permission revoked	Reject protected calls	Attention needed; reconnect through supported account flow
Endpoint or database outage	Fail with correlation ID; do not fake receipt	Restore service then reconcile submission/intent
Crash before submission	Lease expires; preserve known artifacts	Existing explicit retry policy after uncertainty review
Submission acknowledged but response lost	Duplicate key finds durable receipt	Retry identical envelope or read submission status
Changed result under same key	Reject IDEMPOTENCY_CONFLICT	Operator examines conflicting attempts
Source or base edited while running	Reject stale commit	Preserve output; new intent against current inputs
Context too large	CONTEXT_LIMIT before generation where possible	Narrow approved scope or approve a new coverage method
Partial extraction	Preserve valid and failed evidence separately	PARTIAL coverage; block full-week downstream claim
Broken validator	Record validator defect with version	Fix checker and replay saved draft; no blind rewriting
Unsupported claim or missing clearance	Block applicable gate	Human source verification or narrower supported revision
Optional public research unavailable	Record unperformed framing	Continue only if governing contract permits narrower brief
Gate waiting	No downstream generation intent	Normal review queue; reminders follow team policy
Lost publication response	Do not resend blindly	Reconcile exact provider record before delivery retry
Preference projection unavailable	Serve last verified permitted snapshot or block	Show policy retrieval issue; never invent current rules

Retry discipline

Separate retrying an idempotent submission, replaying a saved generation artifact, retrying a failed validation, and performing new generation. Each has different cost and side effects. Only the backend authorizes retries and records attempt count. A model's retryable suggestion does not override state or budget controls.

Preserve the current explicit retry rule for uncertain execution after lease expiry. An operator reviews artifacts and provider/receipt evidence before a new dispatch. For transient read failures, proposed backoff is 5, 20, then 60 seconds within the remaining lease; for result ambiguity, retry the identical write. Stop after bounded attempts and expose the next action. Do not automatically retry an invalid quote, missing permission, stale input, or contract mismatch.

On cancellation, stop eligible successors, fence result admission, and keep prior outputs. On supersession, link the new intent and old base, then resolve stale downstream dependencies. A completed host schedule or chat message cannot override the backend's actual job/result evidence.

Observability

Record correlation, run/job/submission IDs, actor/executor identity, stage, attempt, lease boundaries, frozen input/context hashes, skill/policy/validator versions, admitted refs, findings, and timestamps. Protect raw artifacts with narrower access than ordinary process events. Do not log lease/auth secrets or unnecessary client dialogue.

Track job queue age, scheduled time to first contact, execution and validation duration, source coverage, result rejection reasons, approval wait, revision count, unsupported claim rate, retrieval gaps, stale commits, duplicate receipt rate, missed wakes, and unresolved operational incidents. Measure editorial acceptance and production rewriting separately.

The backend can observe tool contact and submission. It cannot necessarily distinguish a task that failed to wake from a host pause or a model refusing to call tools without operator inspection. Label that alert Executor contact overdue, not definitively ChatGPT down. check_connection can remain business-read-only while ordinary access logs record contact.

Proposed operational defaults

Alert on eligible editorial work older than two expected hourly wakes plus a configured grace interval. Alert on the weekly run missing its expected discovery contact. Escalate immediately for unauthorized reads/writes, source revocation, cross-tenant attempts, or confirmed evidence corruption. These are pilot alert thresholds, not contractual service levels.

Engineering owns adapter/schema/validator failures. Operations owns service deployment, connection/schedule checks, backups, and incident recovery. Whitney owns editorial decisions and reusable preference activation. Giselle owns production constraints and assigned handoffs. Delegate gate authority explicitly in backend capabilities.

Backup and restoration

Back up PostgreSQL and referenced artifacts as a coherent set, with encryption and tested restore. Record source/version manifests so restoration can verify digests. After restore, invalidate unknown active leases, reconcile receipts and external deliveries, reopen only compatible queued work, and inspect revoked permissions before resuming. Define RPO/RTO with the team after deployment and a recovery drill; do not invent achieved recovery times.

23 Integration migration and release plan

Extend the existing workflow

Implement an MCP adapter over current repository and domain services. Start with compatibility reads and receipt storage in a non-production pilot namespace. Reuse existing source validators, supersession, hashes, Gate 1/2 endpoints, and brief state rather than constructing a parallel content database.

Add reviewed executor routing, frozen context manifests, structured feedback/preference records, submission receipts, and outbox/subscription storage where needed. The thirteen-table schema is the baseline, not a prohibition on justified future migrations; each addition should identify its responsibility and avoid duplicate versions or approval authority. Current private artifacts must become durable enough for hosted operation.

Release sequence

1. Inventory actual stage contracts, source metadata, actor capabilities, switches, active prompt profiles, and hash functions. Define v1 adapter mappings and forbidden transitions.

2. Add schema/contract validation, immutable context/result artifacts, credential mapping, and tenant-scoped read/probe tools. Verify no business mutations from probes.

3. Implement atomic claim, guarded submission, identical retry receipts, stale dependency rejection, cancellation, and explicit uncertainty recovery.

4. Connect the real SQL angle review UI to persisted revision/decision events. Remove reliance on browser-local mock state for the pilot's path.

5. Package orchestration and angle skills with explicit live preference retrieval. Manually test a bounded anonymized packet, then test a real authorized source packet.

6. Run scheduled angle/revision execution on the actual account and deployed endpoint. Record independent backend receipts and quality review.

7. Reuse frozen research/brief capabilities, reconcile their current contract defects, and reach a real Gate 2 decision before adding pack jobs.

8. Implement pack plan/asset contracts and Gate 3 with durable handoff. Add publication only as a separate later scope.

Rollout and rollback

Use a per-tenant, per-stage execution switch. Enable a small allowlisted job family and source batch, with max in-flight and result-size limits. Existing provider workers continue their configured jobs; they must not pick up ChatGPT jobs automatically when a connection fails.

Rollback pauses new claims, cancels/supersedes incompatible intents, revokes or narrows credentials as appropriate, and restores the last compatible skill/policy release. Preserve accepted content and all decisions. Switching executor requires a new or explicitly retried intent with current hashes; do not rerun already admitted output merely to restore service.

For schema upgrades, maintain a tested compatibility window for active jobs or drain them before migration. Reject unknown contract versions rather than best-effort coercing factual content. Preserve raw rejected results so a safe adapter fix can replay them. Never change original source/result bytes to make old validation appear to have passed.

24 Pilot roadmap and completion criteria

Phases and evidence

The roadmap is dependency-based. It is not a promised calendar or authorization to activate external calls.

Phase	Scope	Required evidence to proceed
0 Policy and compatibility	Confirm account mode timezone corpus authority plugin permissions versions and deployment	Reproducible manual access; documented configuration; no unresolved private-data scope
1 Transport and ownership	Probe claim context submit issue receipts using small fixtures	Independent persisted result; idempotency conflict stale lease and restart tests
2 Source to moments to angles	One bounded real authorized source batch	Exact original anchors truthful coverage; Whitney judges value and missed opportunities
3 UI revision loop	One angle revision submitted solely from Content Atelier	Scheduled pickup; saved replacement; exact feedback; Gate 1 version and concurrency checks
4 Persistent preferences	Review candidate rule; activate version; process unseen source	Rule scope traceability rollback and no cross-run recall dependency
5 Accepted production brief	Approved angle frozen research bounded repair and Gate 2	Real checks permissions differentiation and explicit human acceptance
6 Pack and handoff	Reviewed plan distinct drafts and Gate 3	Complete selected assets current manifest usable handoff for Giselle
7 Recurring operations	Hosted cycle late sources missed wakes recovery and backup	Repeated weekly execution and a restoration drill with truthful statuses
Optional events	Custom MCP event subscription and fallback polling	Actual account unattended event write plus deduplication expiry and outage evidence

Proposed quantitative pilot thresholds

Before wider operation, require zero unauthorized approval/publication, zero admitted fabricated quotation, zero stale overwrite, and zero duplicate business result in the tested adversarial cases. Require 100 percent of admitted evidence references to resolve to their pinned source versions. A factual zero-defect test does not prove no defects exist in all future work.

Proposed reliability baseline: twenty scheduled trial runs, including at least three recurring weekly cycles, with at least nineteen valid admissions for eligible work and every failed/uncertain run visible and recoverable. Report on-time behavior separately; no-work wakes are not successful generation samples. These are recommended pilot thresholds to approve, not existing achievements.

For editorial quality, establish a founder-reviewed baseline on calibration and unseen sources. Report approval without major reframing, gate review time, missed opportunities, and production rewriting.

Whitney should set targets after reviewing that baseline.

Open decisions with owners

Decision	Owner	When required
WIA timezone source window late-arrival and review day	Whitney and operations	Before installing weekly cadence
Confirmed transcript accounts and use permissions	Source owner and operations	Before real source transfer
Stable hosted endpoint and recovery policy	Engineering and operations	Before unattended recurring pilot
Current handbook profile and comparison cutoff	Whitney with engineering calibration	Before applicable Gate 1 approval
Preference promotion authority and conflict scope	Whitney	Before active preference snapshots
Whole-week capacity or approved partitioning policy	Whitney and engineering	Before large source batches
Pack quantities channels and assigned production work	Whitney and Giselle	Before real pack intents
Permitted account identity and distribution model	Account owner and operations	Before wider organizational rollout
Required revision latency and executor fallback	Product owner	After pilot measurements
Source retention deletion and public clearance policy	Business/source owner	Before sustained client processing

The initial goal is complete when a source batch becomes validated moments and reviewable angles, Whitney requests a revision in Content Atelier, the scheduled executor saves a current replacement with its evidence and feedback, and Gate 1 records an exact reviewed decision. Full proposed production scope is complete only when this extends through an accepted brief, complete approved pack, authorized handoff, recurring recovery, and recorded actual use. A mechanics pass, a mock screen, or a generated draft alone cannot meet those criteria.

Appendix A Worked angle revision example

The identities and imagery statements below illustrate the contract; they are not claims about an actual recording or approved client material.

1. The backend admits TR-101 version 1 and freezes BATCH-042. Its passage P-07 discusses WIA asking a wedding business to assess imagery choices. The evidence supports a collaborative recommendation, with no verified launch or booking result.

2. Extraction admits M-201 with original quote ranges and context. Angle generation saves A-301 revision 1, central point Your campaign imagery is a positioning decision, with WARM as its primary pillar and a source-supported contribution.

3. Whitney requests: Focus more on how WIA helps clients think through imagery decisions. Keep it relevant to wedding business owners. The browser sends base_revision 1, its exact hash, REFRAME, and an idempotency key.

4. One transaction records FB-021 and queues JOB-501. The frontend shows Revision queued and the original angle. A duplicate click returns the same request and job.

5. The hourly worker claims JOB-501, receives its lease, and retrieves context containing revision 1, literal feedback, M-201/P-07, preferences version 12, strategy/offer versions, prior comparisons, and allowed changes.

6. The skill returns a proposed central point explaining WIA's questions and collaborative reasoning. It cites the same moment and does not claim bookings increased. change_summary identifies the framing change and any unresolved request.

7. The backend stores the submission, validates exact evidence and current hashes, saves A-302 superseding A-301 as logical revision 2, and queues applicable comparison checks. Its receipt confirms stored content, not approval.

8. Whitney compares versions and approves revision 2. Only its row/hash authorizes the brief intent. A repeated approval returns the original decision. If she had edited before the submission, JOB-501 would have returned STALE_INPUT and no current replacement would be admitted.

9. The resolved feedback becomes a candidate preference about showing WIA's collaborative reasoning, with its original scope. It becomes active only after explicit preference review; approval of this one angle does not prohibit technical education.

Appendix B Normative contract types and payloads

Scalar and shared types

This catalogue completes the v1 application contracts in Sections 15 to 17. All listed fields are required; a nullable field accepts null explicitly. Arrays may be empty only where stated. Additional properties are rejected. A future schema revision must identify changes rather than silently widening these types.

```
ID: string 1..128; ASCII letters digits underscore hyphen only
Secret: opaque string 16..512; never displayed or logged
Hash: string matching ^sha256:[0-9a-f]{64}$
UTC: RFC3339 timestamp with Z suffix
Text: nonblank Unicode string 1..8000; NUL forbidden
LongText: nonblank Unicode string 1..50000; NUL forbidden
Queue: DISCOVERY | EDITORIAL
Cursor: opaque string 1..2048
ProgressStep: LOAD_CONTEXT|EXTRACT|GENERATE|REFINE|RESEARCH|SUBMIT|
              AWAIT_VALIDATION
IssueCode: CONTEXT_LIMIT|DEPENDENCY_BLOCKED|CONTRACT_MISMATCH|
           VALIDATION_FAILED|SERVICE_UNAVAILABLE|RATE_LIMITED|
           LEASE_EXPIRED|STALE_INPUT|NEEDS_HUMAN_INPUT
RecordRef: {kind: transcript|moment|angle|brief|pack|asset|dossier,
            id: ID, revision: integer >=1, content_hash: Hash}
EvidenceRef: {source_id: ID, source_version: integer >=1,
              passage_id: ID, start: integer >=0, end: integer >start,
              quote: LongText|null,
              role: SUPPORT|CONTEXT|QUALIFICATION,
              claim_status: ClaimStatus, clearance_id: ID|null}
ClaimStatus: PROPOSED|RECOMMENDED|IMPLEMENTED|REPORTED_OUTCOME|
             VERIFIED_OUTCOME|EDITORIAL_INFERENCE
CTA: {wording: Text, destination: string 1..2048|null,
      offer_record_id: ID|null, owner_id: ID|null}
Gap: {gap_id: ID, question: Text, missing_refs: RecordRef[],
      blocking: boolean, owner_id: ID|null}
Coverage: {status: COMPLETE|PARTIAL|NOT_APPLICABLE,
           manifest_hash: Hash|null,
           processed_source_refs: RecordRef[],
           omitted_source_refs: RecordRef[], reasons: Text[]}
```

LongText and whole-result byte limits both apply. Set lower format limits in the pinned plan. A URL-looking destination must pass the backend's URL/domain/offer policy; the model cannot establish destination validity by using a well-formed string. Source permission can be inherited when clearance_id is null, but the server must resolve and verify it.

Exact context packet

ContextPacket contains schema_version literal atelier.context.v1, context_id, job_id, input_hash, created_at, instructions (LongText), required_skill, policy_versions, preference_snapshot {version,digest,markdown}, source_manifest {manifest_hash,source_refs,coverage_policy_version,omissions}, inline_source_segments (SourceSegment[]), base_entity (RecordRef|null), base_payload (typed result payload|null), feedback (Feedback[]), prior_revision_refs (RecordRef[]), history_refs (RecordRef[]), dossier (Dossier|null), pack_plan (PackPlan|null), limits, gaps (Gap[]), allowed_result_type, and allowed_progress_steps (ProgressStep[]). context_hash is computed over the canonical packet excluding context_hash itself and returned separately.

Feedback is {feedback_id, base_ref: RecordRef, actor_id, created_at: UTC, request_type, text: Text, scope: Text, reuse_eligible: boolean}. SourceSegment is {source_ref: RecordRef, passage_id: ID, start, end, text: LongText, speaker: Text|null, attribution_status: VERIFIED|UNVERIFIED|UNKNOWN, start_seconds: number>=0|null, end_seconds: number>=start_seconds|null}. source_ref.kind must be

transcript; text must match its original range.

Generic policy_versions and limits use the exact job-record structures. Omission entries are Gap records, not informal assertions that context is complete. base_payload is a discriminated payload from this catalogue using the base entity kind. instructions are server policy; text inside source/feedback fields remains data.

EvidenceHit is {source_ref: RecordRef, passages: SourceSegment[1..50], score: finite number|null, score_basis: SEMANTIC|FULL_TEXT|HYBRID}. Source-segment responses return one segment per requested passage and preserve request order. Search responses contain up to the requested limit of permitted hits and an opaque next cursor; an exhausted completed search returns null. Progress accepts only allowed_progress_steps; issue reports use IssueCode, summary Text, missing_refs RecordRef[], and advisory retryable boolean. Numeric scores are backend retrieval values rather than model estimates.

Generated payload catalogue

The model uses client_key IDs local to one submission for new objects. The backend allocates saved IDs and returns a client_key to RecordRef mapping. Existing IDs can only be referenced where supplied in context. New revisions are one object, not an array of unrelated candidates.

```
Moment:
 {client_key: ID, insight: Text, moment_type: EXPERTISE|PROCESS|
  COLLABORATION|AUDIENCE_INSIGHT|OFFER_VALUE|OUTCOME|INTERNAL,
  context_summary: Text, audience_relevance: Text,
  wia_contribution: Text|null, qualifications: Text[],
  attribution_notes: Text[], evidence: EvidenceRef[1..50],
  sensitivity: PUBLIC_CANDIDATE|ANONYMIZE|RESTRICTED,
  gaps: Gap[]}

Angle:
 {client_key: ID, title: string 1..200, central_point: Text,
  intended_audience: Text, reader_takeaway: Text,
  strategic_purpose: Text, primary_pillar: ATTRACT|WARM|CONVERT,
  selection_reason: Text, source_demonstration: Text,
  wia_contribution: Text|null, moment_refs: RecordRef[1..50],
  claim_boundaries: Text[], development_direction: Text,
  disposition: SELECT|REFRAME|HOLD|INTERNAL|SKIP,
  suggested_cta: CTA|null, gaps: Gap[]}

Claim:
 {claim_id: ID, wording: Text, status: ClaimStatus,
  evidence: EvidenceRef[1..50], qualifications: Text[],
  public_use: CLEARED|ANONYMIZE|NOT_CLEARED}

Dossier:
 {client_key: ID, angle_ref: RecordRef,
  evidence: EvidenceRef[1..200], allowed_claims: Claim[],
  prior_content_refs: RecordRef[],
  differentiation_notes: Text[], research_queries: Text[],
  public_sources: PublicSource[], gaps: Gap[], coverage: Coverage}

PublicSource:
 {source_key: ID, url: string 1..2048, title: Text,
  retrieved_at: UTC, framing_excerpt: Text,
  verification_status: FETCHED|UNVERIFIED}

Brief:
 {client_key: ID, angle_ref: RecordRef, dossier_ref: RecordRef,
  reader_promise: Text, core_insight: Text,
  message_spine: Text[1..20], narrative: LongText,
  claims: Claim[], prior_difference: Difference[],
  voice_direction: Text, cta: CTA|null,
  format_directions: FormatDirection[1..20],
  creative_notes: Text[], accessibility_notes: Text[],
  questions: Gap[], exclusions: Text[]}

Difference:
 {prior_ref: RecordRef, relationship: DISTINCT|REFRESH|CONSOLIDATE|
  REPEAT|UNKNOWN, explanation: Text}

FormatDirection:
 {format: EMAIL|CAROUSEL|REEL|CAPTION,
  purpose: Text, direction: Text, constraints: Text[]}

PackPlan:
 {plan_id: ID, revision: integer >=1, week_id: ID,
  approved_brief_refs: RecordRef[1..50], slots: Slot[1..30],
  plan_hash: Hash}

Slot:
 {slot_id: ID, order: integer >=1, format: EMAIL|CAROUSEL|REEL|CAPTION,
```

```
  channel: Text, brief_ref: RecordRef, owner_id: ID|null,
  purpose: Text, constraints: Text[], caption_modes: (LONG|SHORT)[],
  due_at: UTC|null}

Asset:
 {client_key: ID, slot_id: ID, brief_ref: RecordRef,
  format: EMAIL|CAROUSEL|REEL|CAPTION, body: AssetBody,
  claim_ids: ID[], production_notes: Text[], gaps: Gap[]}

AssetBody is exactly one format shape:
 EMAIL {subject: string 1..200, preheader: string 0..300|null,
        body: LongText, cta: CTA|null}
 CAROUSEL {slides: Slide[1..20], captions: Caption[0..2]}
 REEL {script: ScriptBeat[1..30], duration_seconds: number>0|null,
       duration_basis: ESTIMATE|MEASURED|UNKNOWN,
       captions: Caption[0..2]}
 CAPTION {mode: LONG|SHORT, text: LongText, cta: CTA|null}
Slide {order: integer >=1, copy: Text, visual_notes: Text,
       accessibility_text: Text|null}
ScriptBeat {order: integer >=1, spoken_copy: Text,
            shot_notes: Text|null}
Caption {mode: LONG|SHORT, text: LongText, cta: CTA|null}

PreferenceCandidate:
 {client_key: ID, wording: Text, rationale: Text,
  scope: {audiences: Text[], pillars: (ATTRACT|WARM|CONVERT)[],
          offers: ID[], formats: (EMAIL|CAROUSEL|REEL|CAPTION)[],
          campaign_ids: ID[], actions: JobType[]},
  feedback_ids: ID[1..50], decision_refs: RecordRef[1..50],
  counterexample_refs: RecordRef[], valid_until: UTC|null}
```

RecordRef.kind for a dossier is dossier with its frozen artifact version/hash; the artifact is indexed by the backend rather than inferred by the model. The dossier/brief types must preserve any current stricter runtime fields through a documented adapter/profile release. Their introduction does not silently replace the existing brief production contract.

Exact result discriminators

MOMENTS payload is {moments: Moment[0..200], coverage: Coverage}; zero valid moments is allowed with truthful COMPLETE or PARTIAL coverage. ANGLES is {angles: Angle[0..200], coverage: Coverage}. ANGLE_REVISION is {angle: Angle, resolved_feedback_ids: ID[], unresolved_feedback_ids: ID[]}; both feedback arrays together must partition the supplied IDs exactly.

DOSSIER is {dossier: Dossier}. BRIEF is {brief: Brief}. BRIEF_REVISION is {brief: Brief, resolved_feedback_ids: ID[], unresolved_feedback_ids: ID[]}. PACK_ASSETS is {plan_id: ID, plan_hash: Hash, assets: Asset[1..30]}. ASSET is {plan_id: ID, plan_hash: Hash, asset: Asset}. PACK_REVISION is {base_manifest_hash: Hash, plan_id: ID, plan_hash: Hash, assets: Asset[1..30], resolved_feedback_ids: ID[], unresolved_feedback_ids: ID[]}. PREFERENCE_CANDIDATES is {candidates: PreferenceCandidate[0..50]}.

Moment references in ANGLES point only to admitted moments. Angle refinement must retain valid moment lineage or explicitly justify an authorized evidence change. Asset claim_ids resolve only to the governing brief's allowed claims. Pack outputs must cover the selected target slots exactly once, in plan order, and preserve required caption modes. Order fields are unique contiguous integers starting at one. Arrays of empty text do not satisfy content requirements.

Receipt and finding types

Submission status is RECEIVED, VALIDATION_PENDING, ADMITTED, REJECTED, or STALE. Receipt fields are submission_id, receipt_id, result_hash, status, admitted_refs {client_key,record_ref}[], validation_findings Finding[], duplicate boolean, and received_at UTC. A pending receipt may have empty admitted_refs; ADMITTED must correspond to durable business rows or a truthful empty discovery result. Check_connection and tool success fields use the catalogue in Section 17; include

schema_version for each tool's registered output schema at implementation.

Finding is {finding_id: ID, rule_id: ID, severity: BLOCKER|WARNING|INFO, field_path: string 1..512, evidence_ids: ID[], description: Text, suggested_action: Text, validator_version: ID, assessed_hash: Hash, source: NORMALIZER|DETERMINISTIC|SEMANTIC|HUMAN}. report_job_progress step is an allowed stage-specific enum; completed_units and total_units are both null when no meaningful measured units exist, otherwise nonnegative integers with completed<=total. Reported units are checked against known work.

Contract version compatibility is fail-closed. Before deployment, encode this catalogue in registered MCP input/output schemas and shared backend validators. Run examples and negative fixtures against that executable schema. These definitions are the specification; this document does not claim the schema or adapter has been deployed.

Appendix C Acceptance and adversarial test catalogue

Case	Required assertion
Scheduled real origin	Independent backend receipt and correlated task evidence identify actual execution
Tab closed	Scheduled processing works with the tab closed; full app exit is separately tested
Probe preflight	Connection read changes no business state; missing access produces visible failure
Two simultaneous claimers	One current lease and one active attempt; losing caller gets NO_WORK
Duplicate result after lost reply	Original receipt returned; one business row set and no duplicate successor
Same key different result	Conflict rejected; original accepted data preserved
Wrong job or tenant token	Authorization/lease guard rejects without data disclosure
Stale edit during generation	Saved newer version survives; generated stale artifact cannot become current
Cancel during generation	Commit guard fences result and successors
Revoked clearance after context read	Admission and gate checks reject prohibited use
Missing source or invalid quote	No invalid evidence admitted; gap shown with exact source target
Source correction or duplicate capture	Source lineage preserved; recurrence not inflated
Whole-week oversized input	Explicit CONTEXT_LIMIT; no silent summary partition or completeness claim
Partial extraction	Accepted and rejected sets preserved; coverage visible; policy controls downstream work
Prompt injection in transcript	No arbitrary access approval publication or policy change
Same argument new title	Comparison identifies central-point overlap using eligible family history
Changed CTA or evidence	Old approval hash no longer authorizes downstream generation
Missing or unperformed research	Unknown coverage remains explicit; no fabricated public source
Citation digits checker defect	Label digits not treated as quantities; actual numbers still checked

Case	Required assertion
Brief repair budget	One permitted repair; remaining defect reaches review/block state
Mock versus live	Different browser/device sees same committed live decision; mock state is labelled
Preference one-off correction	Candidate retains original scope; no automatic universal rule
Preference rollback	New activation points to reviewed snapshot; history and old job pins remain
Memory unavailable	Job succeeds with explicit backend context or reports missing mandatory input
Cross-client recalled material	Output review detects contamination; backend IDs cannot cross tenants
Restart after artifact saved	Replay reconciles hash and commit without another generation
Lease expired after long generation	Stale result rejected; explicit uncertainty recovery used
Missing pack slot or caption	Gate 3 blocked; no silent reduction in selected plan
Publication response lost	Reconcile provider state before any resend
Event duplicated or out of order	Idempotent event and claim handling; reload current backend state
Event and hourly wake overlap	One admitted result and no feedback loop
Subscription expired or outage	Polling finds queued work; operator receives actionable alert
Backup restore	SQL/artifact digests agree; unknown leases invalid; external receipts reconciled

Use offline deterministic fixtures for contract and state checks, authorized account tests for scheduling/permissions, and founder review of real and unseen recordings for editorial quality. Record sample size and failures. Do not count prepared requests, undispatched calls, or cached replays as fresh scheduled generation.

Appendix D Sources and evidence register

Internal sources

[S0] Content Approval Gates, ChatGPT conversation 6acf12c7-35b0-83ee-a295-56f7435be04a. User-selected architecture and subsequent memory/project preference statements, retrieved 14 October 2026. The preserved responsibilities, two proposed schedulers, three gates, backend memory principle, and revision flow originate here. Assistant entries returned as content-reference markers; the user-selected text supplies the accessible architecture passage.

[S1] Content Atelier project_agenda.md, updated 14 October 2026. Local project document at content_atelier/project_agenda.md in the Wedding_Industry_Ads workspace. Primary source for purpose, roles, audience, strategy, twelve-step process, current implementation boundaries, open decisions, and delivery agenda.

[S2] Executed tests and ChatGPT observations, TEST_RESULTS.md, experiment 14 October 2026. Standalone MCP write test output in the Codex project Build standalone MCP write test. Reports actual local/remote/manual/scheduled tests and links raw receipts/audit evidence. This architecture used the report and did not rerun the experiment or independently revalidate every linked evidence file.

[S3] Content Atelier compact database schema, approved design 9 October 2026, content_atelier/database-schema-compact.md. Primary contract for thirteen tables, two vectors, JSON validation, hashes, supersession, gate bindings, job/event ownership, and current implementation caveats. Later brief implementation records supersede its older downstream status statements.

[S4] Content brief implementation status, 12 October 2026 with subsequent W1-3 and 13 October buying-research pilot updates, content_atelier/implementation-status-content-briefs.md. Source for implemented research/brief controls, bounded repair, gate blockers, pilot-only cutoff, cache/cost distinctions, and separate production scope.

[S14] backend/backend_pipeline/repository.py and store.py, local read 14 October 2026. The claim/guard/heartbeat implementation uses queued/running/completed states, lease tokens, row locking, and explicit retry after expiry. store.py supplies the canonical JSON and UTF-8 SHA-256 implementation. Read-only inspection supports the migration guidance; no code, database, or executor was changed for this document.

Official OpenAI sources

All official pages below were retrieved on 14 October 2026. Availability and account-specific configuration must be checked again before activation. The design separates official product descriptions from observed account behavior and proposed Content Atelier controls.

ID	Source and supported topic
S5	Plugin architecture Skills MCP structured results and optional UI
S6	Skills Instruction resources activation and tool boundary
S7	Scheduled tasks in ChatGPT Paid cadence connected apps approval pauses project-file restriction
S8	Scheduled tasks Cloud versus local tasks existing-chat context skills and broader cadence guidance

ID	Source and supported topic
S9	Projects in ChatGPT Project-only containment default memory and Work restriction
S10	Memory in ChatGPT Relevance-based memory incomplete retention and controls
S11	MCP Events Custom Work subscriptions protocol 2026-07-28 webhook verification and delivery
S12	Authentication OAuth 2.1 MCP metadata credential/resource boundaries
S13	Build an MCP server Schemas annotations structured results and static skill import
S15	Managing app permissions in ChatGPT Account-dependent approvals risk and app controls
S16	Data controls in ChatGPT Model-improvement controls and account data handling
S17	Security and Privacy Least privilege server validation retention and protected logs

Source precedence

Use current implementation and the newest verification record for what exists; use the agenda and latest authorized founder direction for editorial purpose. Use official documentation for described host capabilities, then test the actual account for availability and behavior. Proposed contracts and operating thresholds in this document are design decisions, not experimentally verified guarantees.

Project-only Work remains unavailable under the retrieved documentation. Custom MCP Events warrants a separate account test; the durable queue, approval gates, and backend memory still apply.